第5章 存储器管理

5.1 存储器管理概述
5.2 连续分配存储管理
5.3 对换
5.4 基本分页存储管理
5.5 基本分段存储管理
5.6 段页式存储管理
5.7 虚拟存储器的基本概念
5.8 请求分页存储管理
5.9 请求分段存储管理

实存管理

虚拟存储管理

5.1 存储器管理概述

- 存储器管理的基本功能
  - 内存分配与回收

    内存分配/回收策略(算法)、内存分配数据结构
  - 内存保护

    硬件检查越界，软件中断处理
  - 地址映射

    逻辑地址、物理地址，重定位
  - 内存扩充

    虚拟存储技术

5.1.1 存储器的层次结构

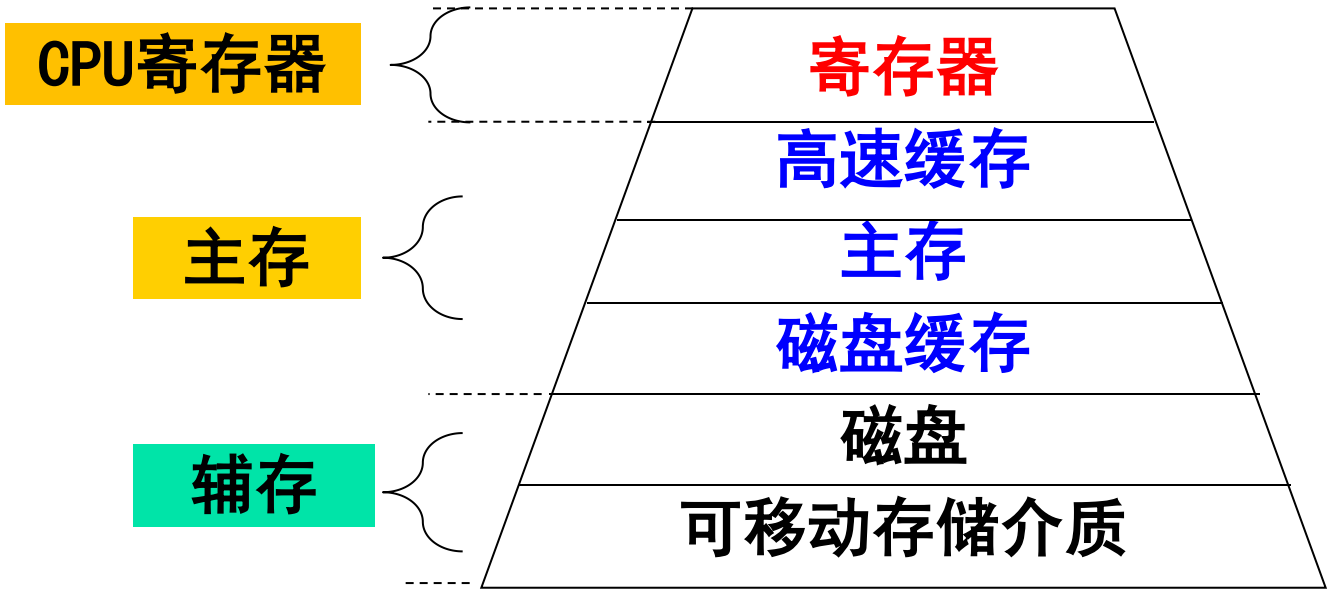


图5-1　计算机系统存储层次示意

5.1.2 逻辑地址与物理地址

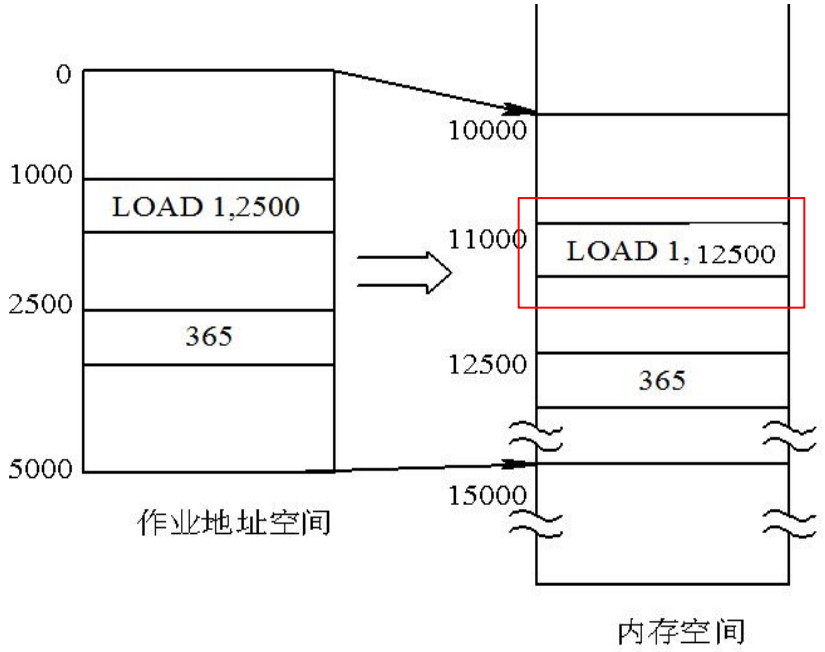
- 逻辑地址
  - 目标程序使用的地址单元称为逻辑地址(相对地址或虚地址)。
  - 一个用户作业的目标程序的逻辑地址集合称为该作业的逻辑地址空间。
  - 一维逻辑地址从0开始编号。二维逻辑地址各段内地址也从0开始。
- 物理地址
  - 主存中的实际存储单元编号称为物理地址(绝对地址或内存地址)。
  - 物理地址的总体构成物理地址空间。

5.1.2 逻辑地址与物理地址

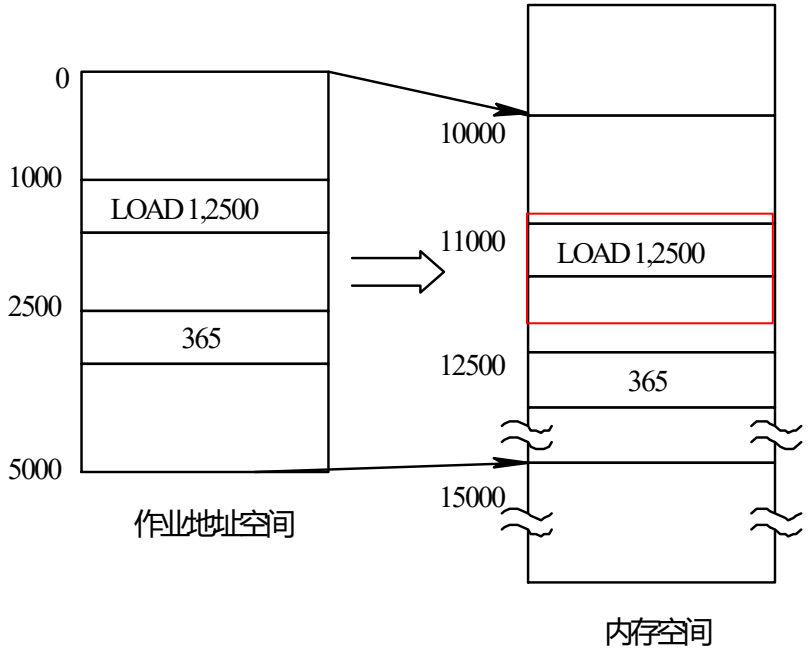
- 重定位
  - 为了保证程序的正确执行，必须把程序和数据的逻辑地址转换为物理地址，这一工作叫做重定位(地址转换，地址映射）。
  - 重定位可分两种：
    - 静态重定位：是在装入一个作业时，把作业中的指令地址和数据地址全部转换成绝对地址。
    - 动态重定位：是装入一个作业时，不进行地址变换，而是直接把作业装入到分配的主存区域中，在作业执行的过程中，每当执行一条指令，都由硬件的地址转换机构转换成绝对地址。

5.1.2 逻辑地址与物理地址

- 重定位



（a）静态重定位



（b）动态重定位

图5-3　重定位

5.1.3 程序的装入和链接

- 用户程序的处理步骤

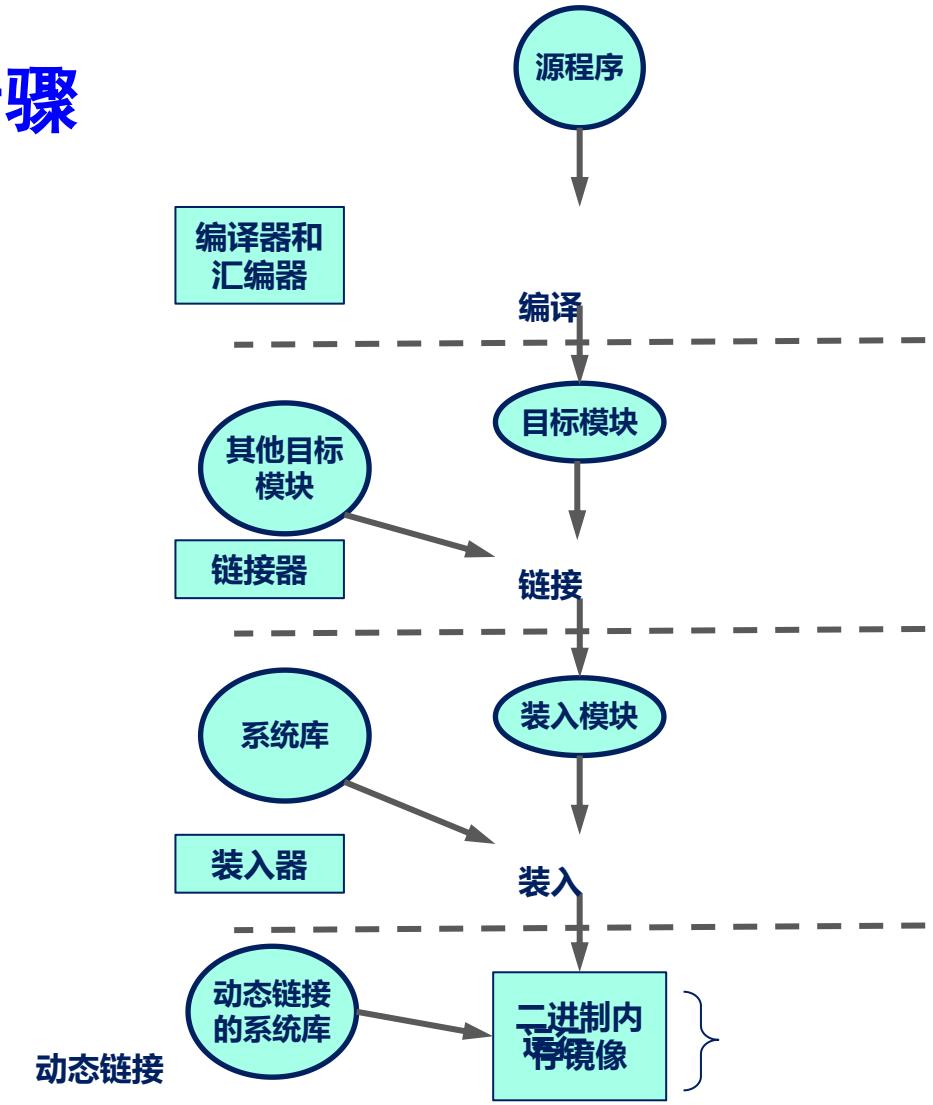


图5-4 用户程序的处理步骤

5.1.3 程序的装入和链接

- 将程序(模块)装入内存时，可以有三种方式：
  - 绝对装入方式
  - 可重定位装入方式(静态重定位方式)
  - 动态运行时装入方式

5.1.3 程序的装入和链接

- **绝对装入（Absolute Loading ）**
  - 绝对装入方式是事先已经知道用户程序入主存后的位置，程序在编译或链接时直接指定固定的物理内存地址，装入时无需重定位，直接加载到预设的固定位置。
  - 特点：
    - 地址固定；无灵活性，不适合多任务；装入过程无需额外计算。
  - 适用场景：
    - 早期单道批处理系统（如DOS单任务环境）。
    - 嵌入式系统中程序固化到ROM的特定地址（如Bootloader），如：编译时指定程序入口地址为 0x7C00（传统BIOS引导扇区地址）。

5.1.3 程序的装入和链接

- **静态重定位装入（Static Relocatable Loading）**
  - 在多道程序环境下，编译程序不能预知用户程序入主存后的位置，因此编译汇编所得到的目标模块的起始地址通常是从0开始的，目标模块中的其它地址也都是相对于起始地址计算的。
  - 静态重定位装入方式是程序在装入时，由装入器Loader对装入模块一次性完成重定位后装入内存。
  - 特点：程序一旦装入内存，若需要移动则必须重新装入并修正地址。

5.1.3 程序的装入

- **动态重定位装入（Dynamic Relocatable Loading）**
  - 动态重定位装入方式是装入程序将装入模块装入内存时，不将装入模块中的逻辑地址修改为物理地址，而是在运行时在硬件支持（如基址寄存器）的辅助下再进行动态重定位。
  - **特点：**
    - 灵活移动：程序在内存中移动时只需更新基址寄存器，无需修改程序代码。
    - 硬件依赖：需要CPU提供基址寄存器等地址转换硬件支持。
  - **实现方式**
    - 物理地址 = 基址寄存器值+逻辑地址

5.1.4 存储管理方式

- 思考：操作系统完成存储器管理功能需要考虑哪几个问题？
  - 问题1：
  - 问题2：
  - 问题3：
  - 问题4：



5.1.4 存储管理方式

- 思考：操作系统完成存储器管理功能需要考虑哪几个问题？
  - 问题1：分配
  - 问题2：回收
  - 问题3：数据结构
  - 问题4：重定位

5.1.4 存储管理方式

- 连续分配方式----- 分区管理方式
  - 固定分区分配
  - 动态分区分配
  - 可重定位分区分配
- 离散分配方式
  - 页式存储管理方式
  - 段式存储管理方式
  - 段页式存储管理方式
- 虚拟存储器
  - 请求分页存储管理方式
  - 请求分段存储管理方式

5.2 连续分配方式

- **思想：**
  - 又称分区管理方式，是指一个用户程序分配一个连续的内存空间。分区是指内存中的一个连续区域。
  - 分区分配方式可分为：
    - 单一连续分配
    - 固定分区分配
    - 动态分区分配
    - 动态重定位分区分配

5.2.1 单一连续分配

- **思想：**
  - 把内存分为系统区和用户区两部分，系统区仅提供给OS使用，通常是放在内存的低址部分。
  - 用户区是指除系统区以外的全部内存空间，提供给用户使用。
  - 这是最简单的一种存储管理方式，但只能用于单用户、单任务的操作系统中。



5.2.2 固定分区分配

- **思想：**
  - 将内存划分为若干个固定大小的区域（分区），每个分区中装入一道作业，允许几道作业并发运行。
  - 当有空闲分区时，便可从后备队列中选择一个作业装入该分区。
  - 思考：前面的几个问题的处理？

5.2.2 固定分区分配

- **数据结构**
  - 分区分配表

分区号	大小(K)	始址(K)	状态
1	32	100	未分配
2	64	132	未分配
3	128	196	未分配
4	128	324	未分配
5	188	452	未分配

图5-1 分区分配表

操作系统
100K
132K
196K
324K
452K

图5-2 内存情况

5.2.2 固定分区分配

- **固定分区分配策略**
  - 当有一用户程序要装入时，由内存分配程序检索该表，从中找出一个能满足要求的、尚未分配的分区，将之分配给该程序，然后将该表项中的状态置为“已分配” 。
  - 若找不到大小足够的分区，则拒绝为该用户程序分配内存。

5.2.2 固定分区分配

- 固定分区分配流程



分区号	大小(K)	始址(K)	状态
1	32	100	已分配
2	64	132	已分配
3	128	196	未分配
4	128	324	未分配
5	188	452	已分配



5.2.2 固定分区分配

- 固定分区回收流程

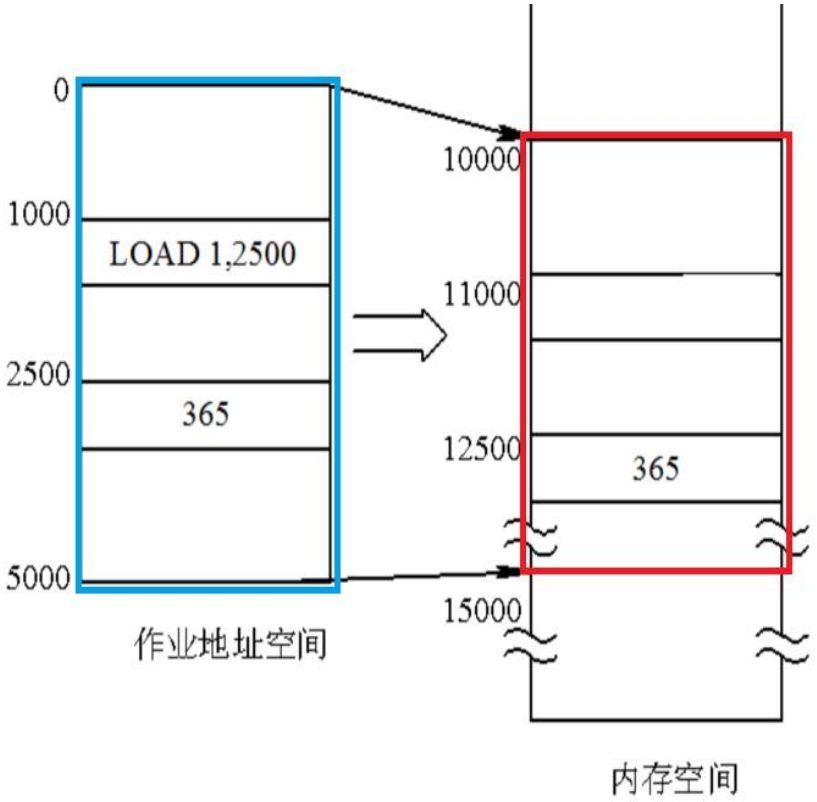
作业A完成 → 作业C完成 → 作业B完成

分区号	大小(K)	始址(K)	状态
1	32	100	已分配
2	64	132	已分配
3	128	196	未分配
4	128	324	未分配
5	188	452	已分配



5.2 连续分配方式

- **重定位**



逻辑地址2500对应的物理地址是？

- **物理地址= 分区起始地址+逻辑偏移地址**

5.2.2 固定分区分配

- **固定分区分配方式的缺点**
  - 大程序可能无法装入
  - 主存空间利用率不高—作业往往不可能恰好填满分区
  - 限制了多道运行的程序数
  - 作业动态扩充主存困难
  - 各分区作业要共享程序和数据也难实现

5.2.3 动态分区分配

- **思想：**
  - 动态分区分配又称可变分区分配，是根据进程的实际需要，动态地为之分配内存空间。
  - 思考：前面的几个问题的处理？

5.2.3 动态分区分配

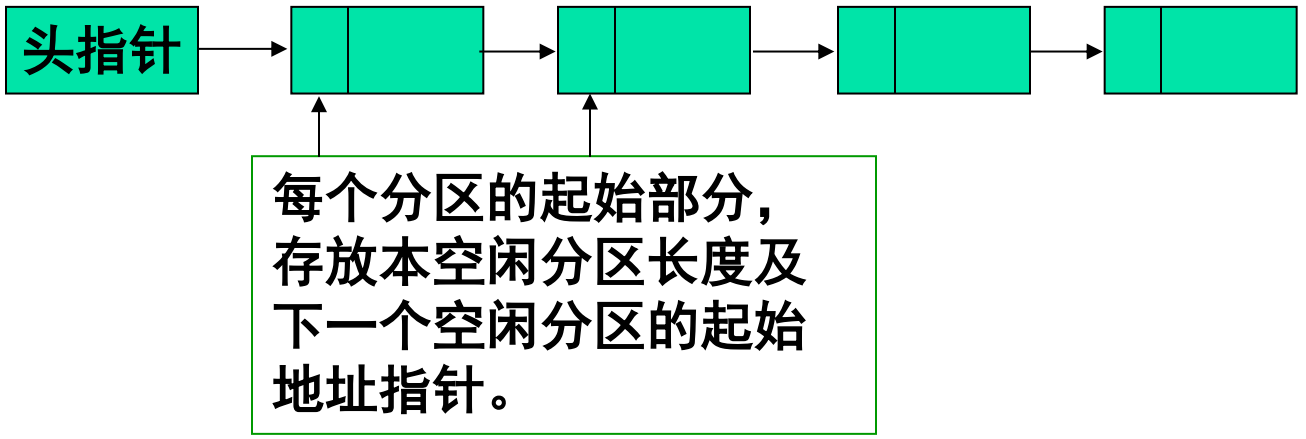
- 数据结构
  - 可以有两种形式：空闲分区表、空闲分区链
  - 空闲分区表：
    - 每个空闲分区占一个表目，表目包括分区号、分区始址、分区大小等

分区号	分区始址	分区长度
1	100KB	128KB
2	800KB	200KB
3	1300KB	700KB



5.2.3 动态分区分配

- 数据结构
  - 空闲分区链：



  - 优点：自身不占用存储空间
  - 缺点：比空闲分区表管理复杂

5.2.3 动态分区分配

- 动态分区分配策略

首次适应算法
(First Fit，FF)

空闲分区表**按地址递增排序**。分配时**从表首开始顺序查找，直至找到一个大小能满足要求的空闲分区**；然后按作业大小**划出**一块内存空间分配给请求者，**余下的**空闲分区仍留在表中。

循环首次适应算法(Next Fit，NF)

由首次适应算法演变而成。为进程分配内存时，不再是每次从表首开始查找，而是从上次扫描结束处开始顺序查找，直至找到一个大小能满足要求的空闲分区，从中划出一块与请求大小相等的分区分配给作业。为实现该算法，应设置一起始查找指针。

最佳适应算法
(Best Fit，Bf)

每次分配时，总是将能满足要求的最小分区分配给请求者。将空闲分区按其容量从小到大顺序排列，分配时从表首开始顺序查找——加快查找。

最坏适应算法
(Worst Fit，)

将空闲分区按其容量从大到小顺序排列，分配时从表首开始顺序查找。

5.2.3 动态分区分配

- 例题：在可变分区存储管理下，按地址排列的内存空闲区为：100KB、500KB、200KB、300KB和600KB。现有若干用户程序，其所需内存依次分别为212KB、417KB、112KB和426KB，分别用首次适应算法、最佳适应算法、最坏适应算法，将它们装入到内存的哪些空闲分区？哪个算法能最有效利用内存？

序号	空闲区大小
1	100KB
2	500KB
3	200KB
4	300KB
5	600KB

5.2.3 动态分区分配

解：采用首次适应算法，空闲分区表为：

序号	空闲区大小
1	100KB
2	500KB
3	200KB
4	300KB
5	600KB

程序请求		空闲区		新空闲区
212KB	→	500KB	⇨	288KB
417KB	→	600KB	⇨	183KB
112KB	→	288KB	⇨	176KB

426KB，无法装入内存

序号	空闲区大小
1	100KB
2	176K
3	200KB
4	300KB
5	183K

5.2.3 动态分区分配

解：采用最佳适应算法，空闲分区表为：

程序请求	空闲区	剩余空闲区
212KB	300KB	88KB
417KB	500KB	83KB
112KB	200KB	88KB
426KB	600KB	74KB

序号	空闲区大小
1	100KB
2	200KB
3	300KB
4	500KB
5	600KB

序号	空闲区大小
1	100KB
2	88k
3	88k
4	83k
5	74k

5.2.3 动态分区分配

解：采用最坏适应算法，空闲分区表为：

程序请求	空闲区	剩余空闲区
212KB	600KB	388KB
417KB	500KB	83KB
112KB	388KB	276KB

426KB，无法装入内存

序号	空闲区大小
1	600KB
2	500KB
3	300KB
4	200KB
5	100KB

序号	空闲区大小
1	276k
2	83k
3	300KB
4	200KB
5	100KB

5.2.3 动态分区分配

- 动态分区分配流程
  - 分配内存过程如图所示
  - 其中：

请求分区大小u. size

分区大小为m. size

不再切割的剩余分区大小为size

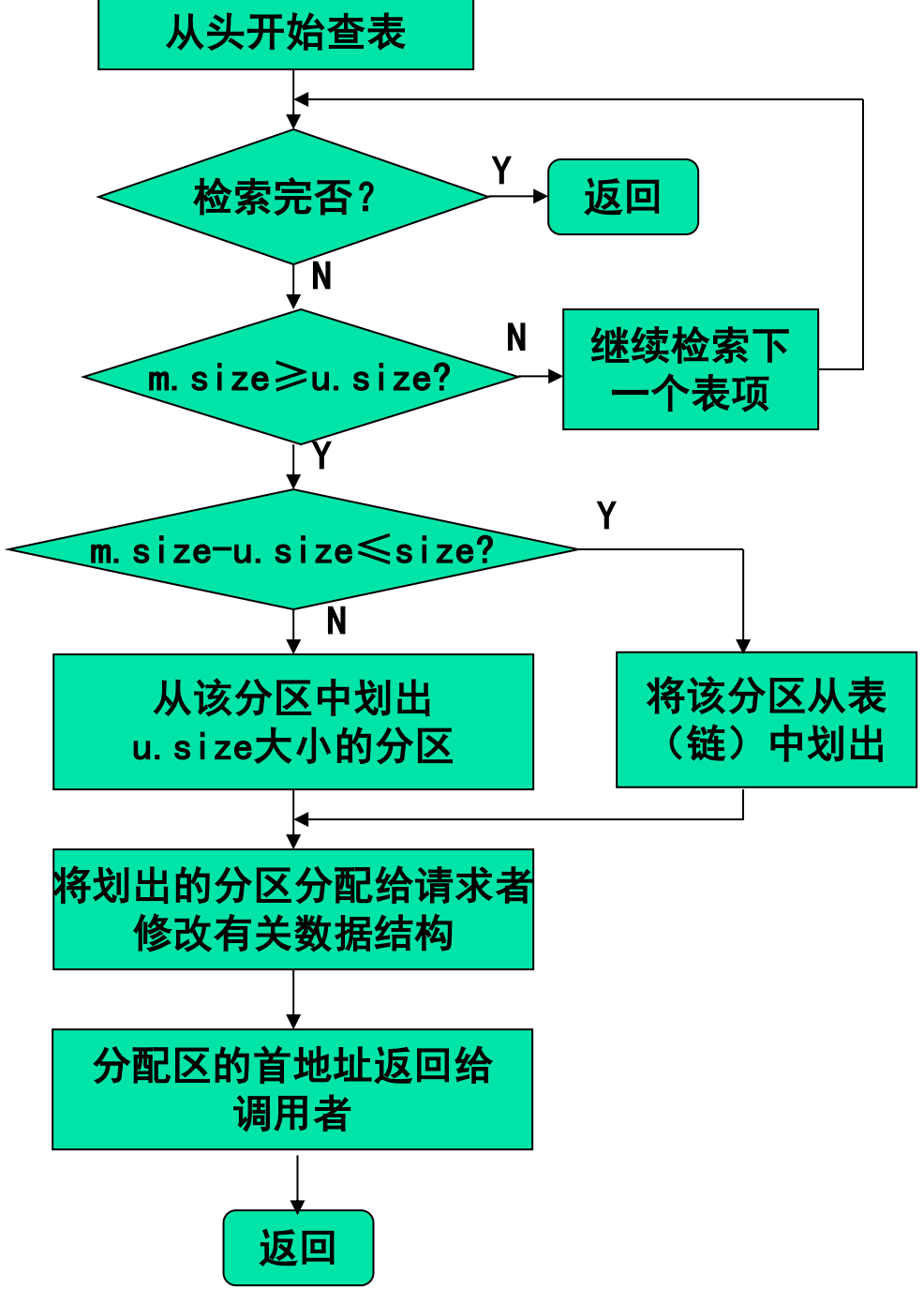


图5-3 动态分区内存分配流程

5.2.3 动态分区分配

- **动态分区分配回收流程**
  - 根据回收区的首地址，在空闲分区表（链）找到插入点，此时可能出现4种情况之一（假设空闲分区表按地址从低到高顺序排列，F1，F2均表示空闲分区）

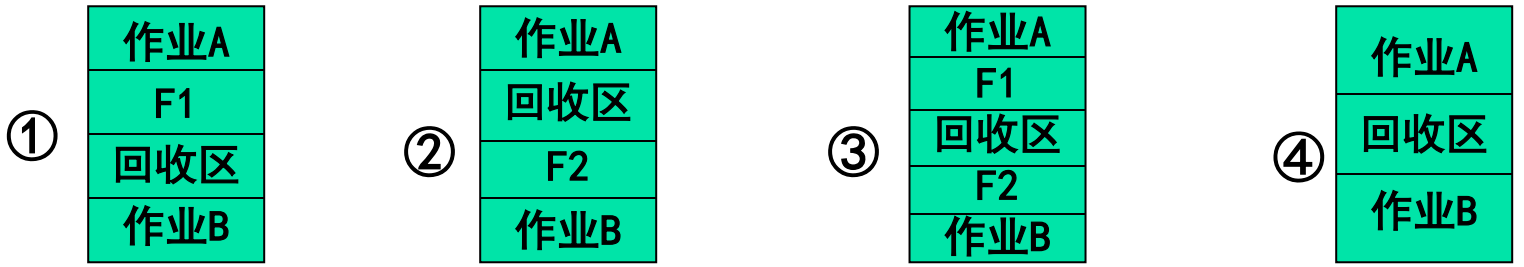


图5-3 内存回收时的情况

  - 思考：上述4种情况下，回收后空闲分区表(链）该如何修改？
  - 分区回收流程（略）

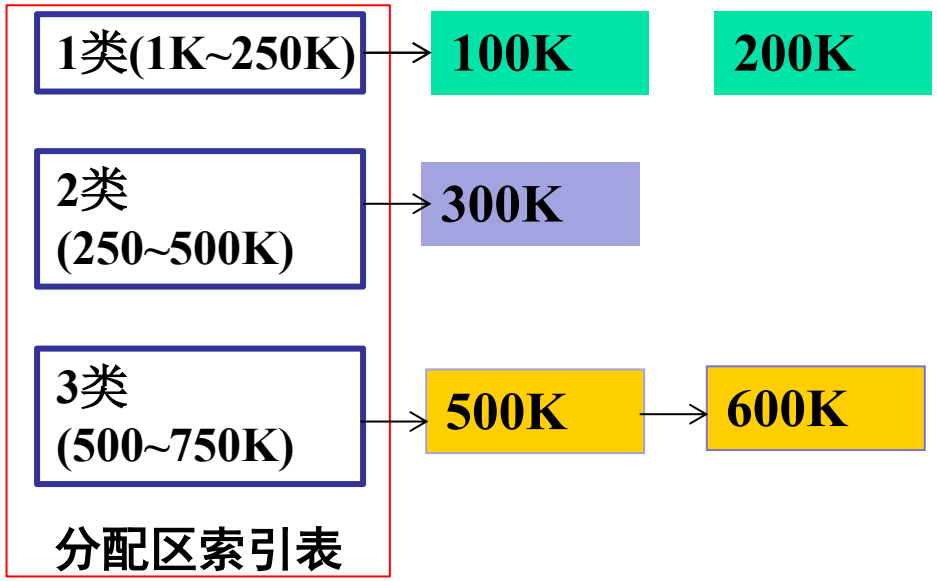
5.2.3 动态分区分配

- 基于索引搜索的动态分区分配算法
  - 基于顺序搜索的动态分区分配算法：搜索效率较低
  - 目标：为了提高搜索空闲分区的速度，在大、中型系统中往往会采用基于索引搜索的动态分区分配算法
  - 分区分配算法：
    - 快速适应算法
    - 伙伴系统
    - 哈希算法

5.2.3 动态分区分配

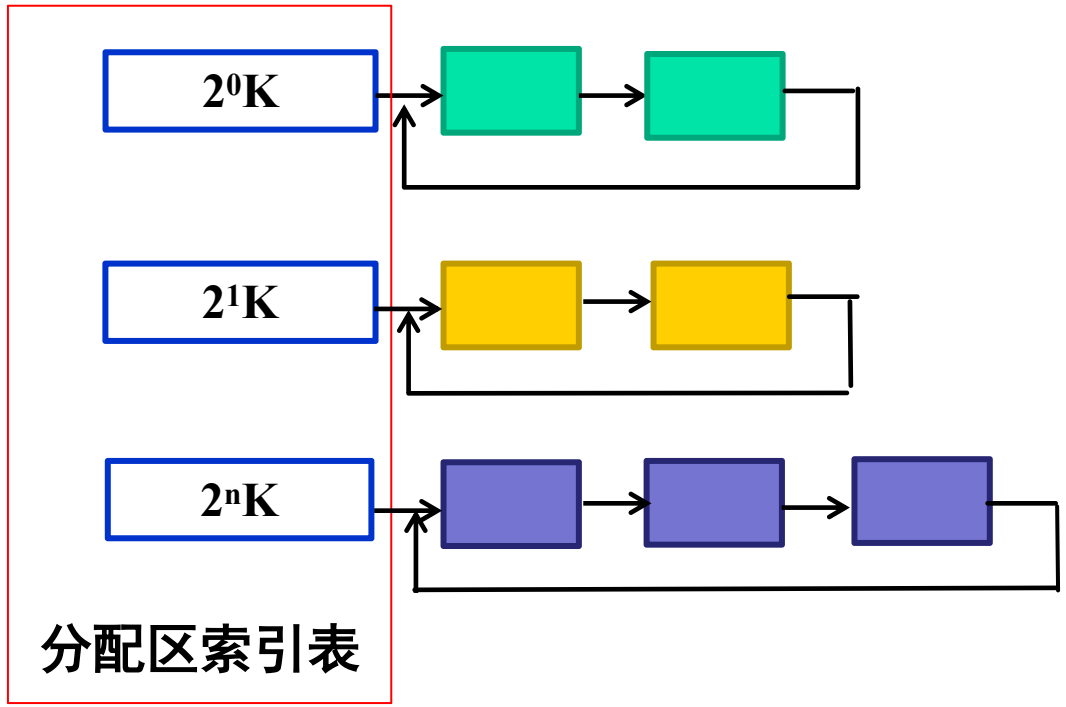
- 基于索引搜索的动态分区分配算法
  - 快速适应算法（Quick fit）

序号	空闲区大小
1	100KB
2	200KB
3	300KB
4	500KB
5	600KB



5.2.3 动态分区分配

- 基于索引搜索的动态分区分配算法
  - 伙伴系统（Buddy System）

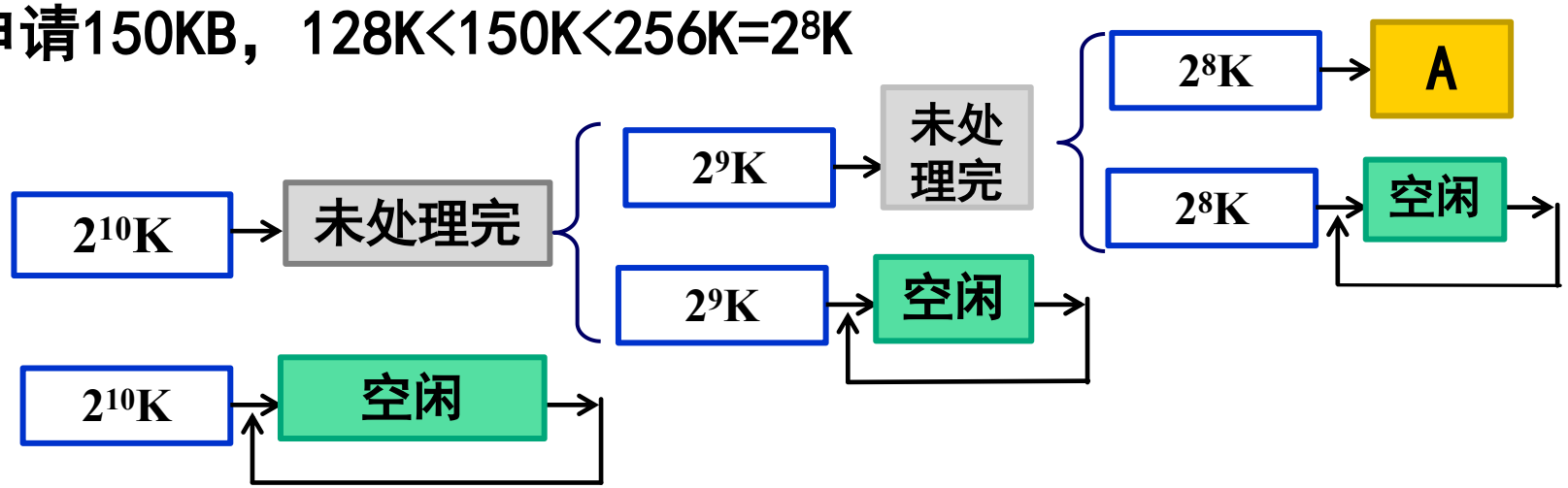


5.2.3 动态分区分配

- 基于索引搜索的动态分区分配算法
  - 伙伴系统（Buddy System）
  - 初始：用户区：2048KB=2^{11}KB

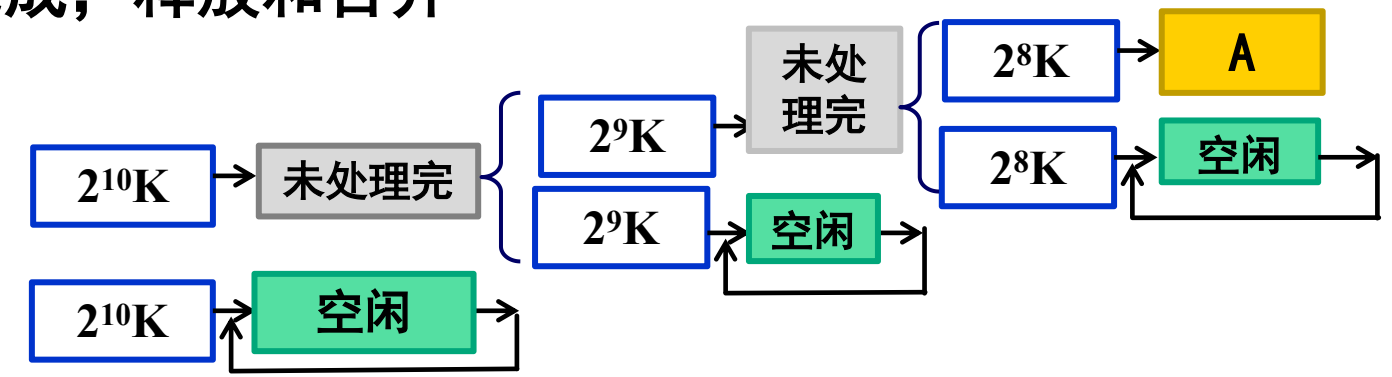


  - 作业A申请150KB，128K<150K<256K=2^8K



5.2.3 动态分区分配

- 基于索引搜索的动态分区分配算法
  - 伙伴系统（Buddy System）
  - 作业A完成，释放和合并

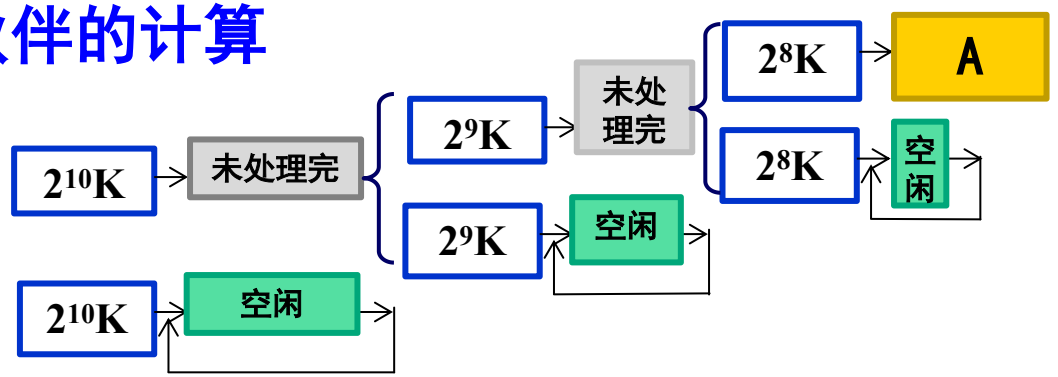


  - 逐步合并，最后：

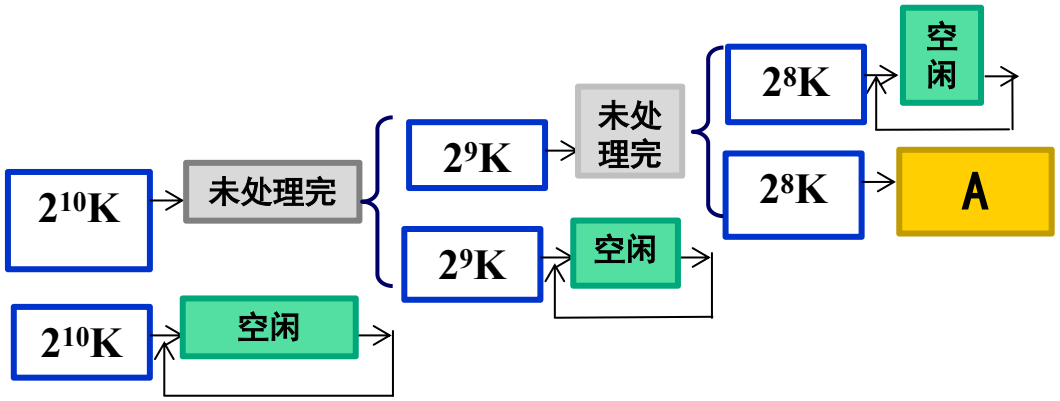


5.2.3 动态分区分配

- 基于索引搜索的动态分区分配算法
  - 伙伴系统（Buddy System）
  - 回收时伙伴的计算



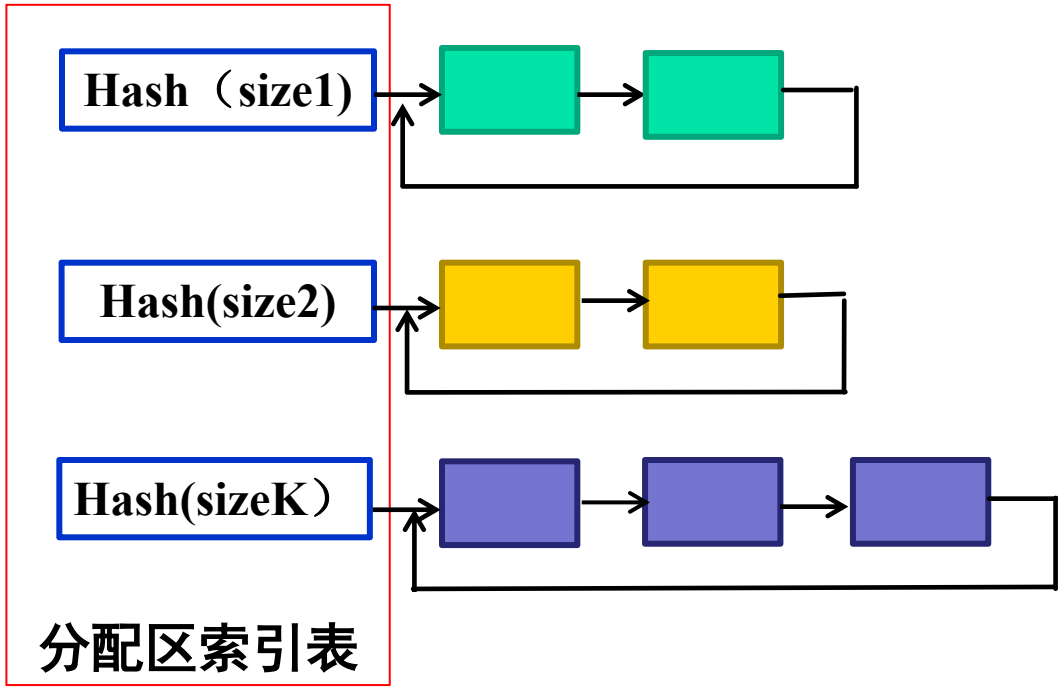
- 或



$$Buddy_k(X)=\begin{cases} x+2^k(\text{若}x\ MOD\ 2^{k+1}=0) \\ x-2^k(\text{若}x\ MOD\ 2^{k+1}=2^K) \end{cases}$$

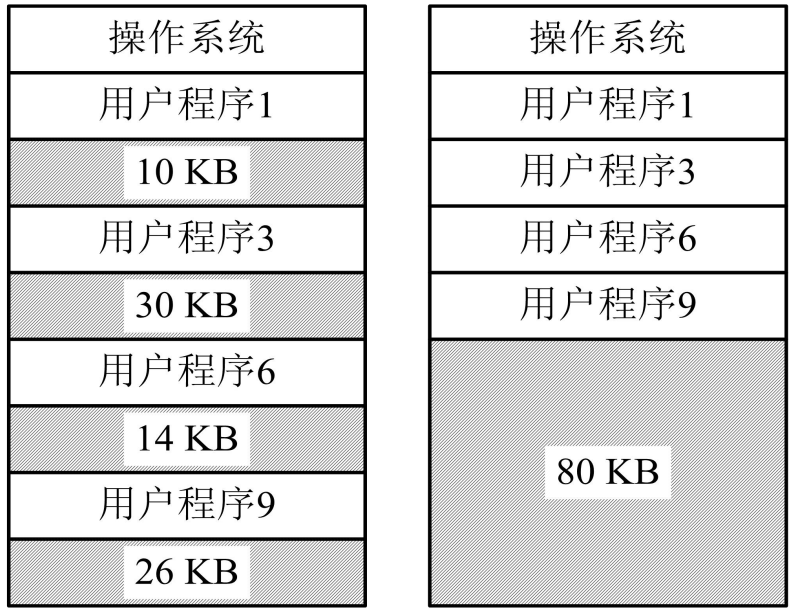
5.2.3 动态分区分配

- 基于索引搜索的动态分区分配算法
  - 哈希算法（Hash）



5.2.4 可重定位分区分配

- **紧凑**
  - 目的：解决内存“碎片”问题
  - 通过移动内存中作业的位置，把原来多个分散的小分区拼接成一个大分区的方法，称为“紧凑”。



(*a*) 紧凑前　　(*b*) 紧凑后

图5-4 紧凑技术

5.2.4 可重定位分区分配

- 动态重定位的实现
  - 专设两个控制寄存器：基址寄存器和限长寄存器
  - 物理地址 = 逻辑地址 + 基址寄存器中的地址值

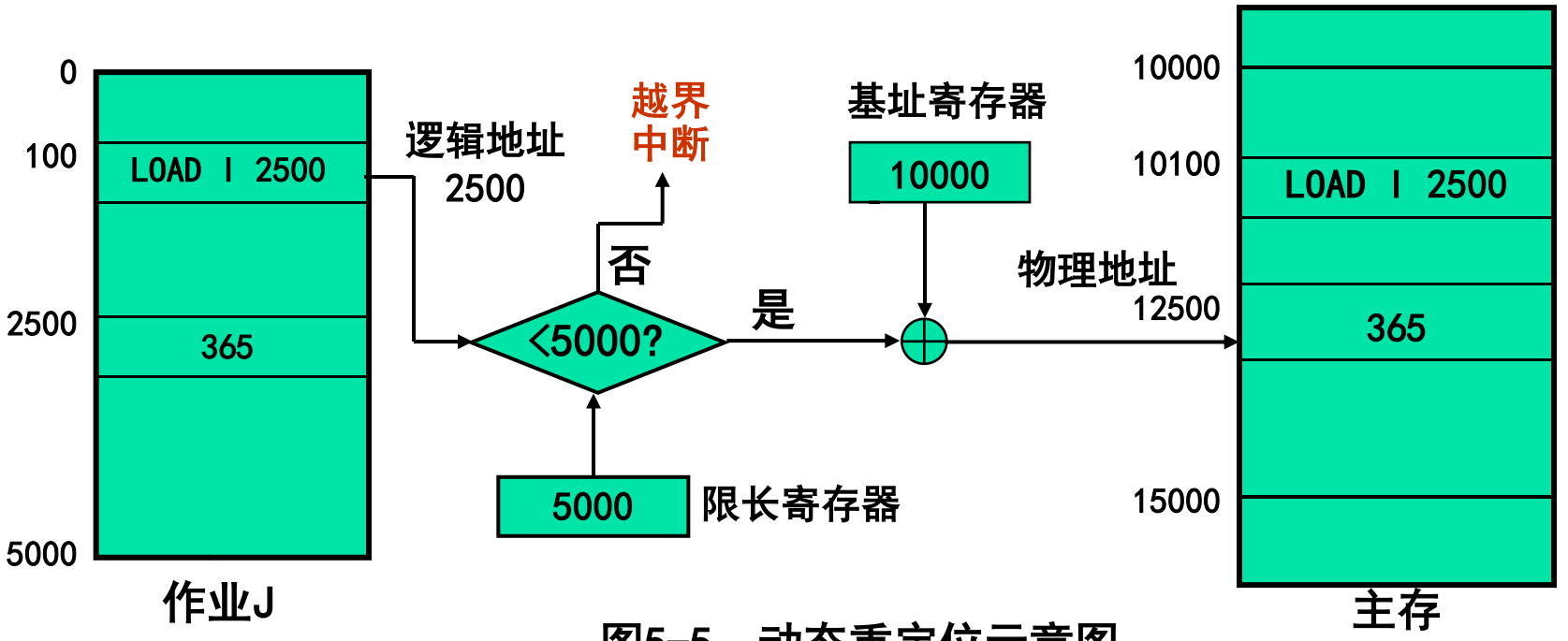


图5-5 动态重定位示意图

5.2.4 可重定位分区分配

- 动态重定位分区分配算法

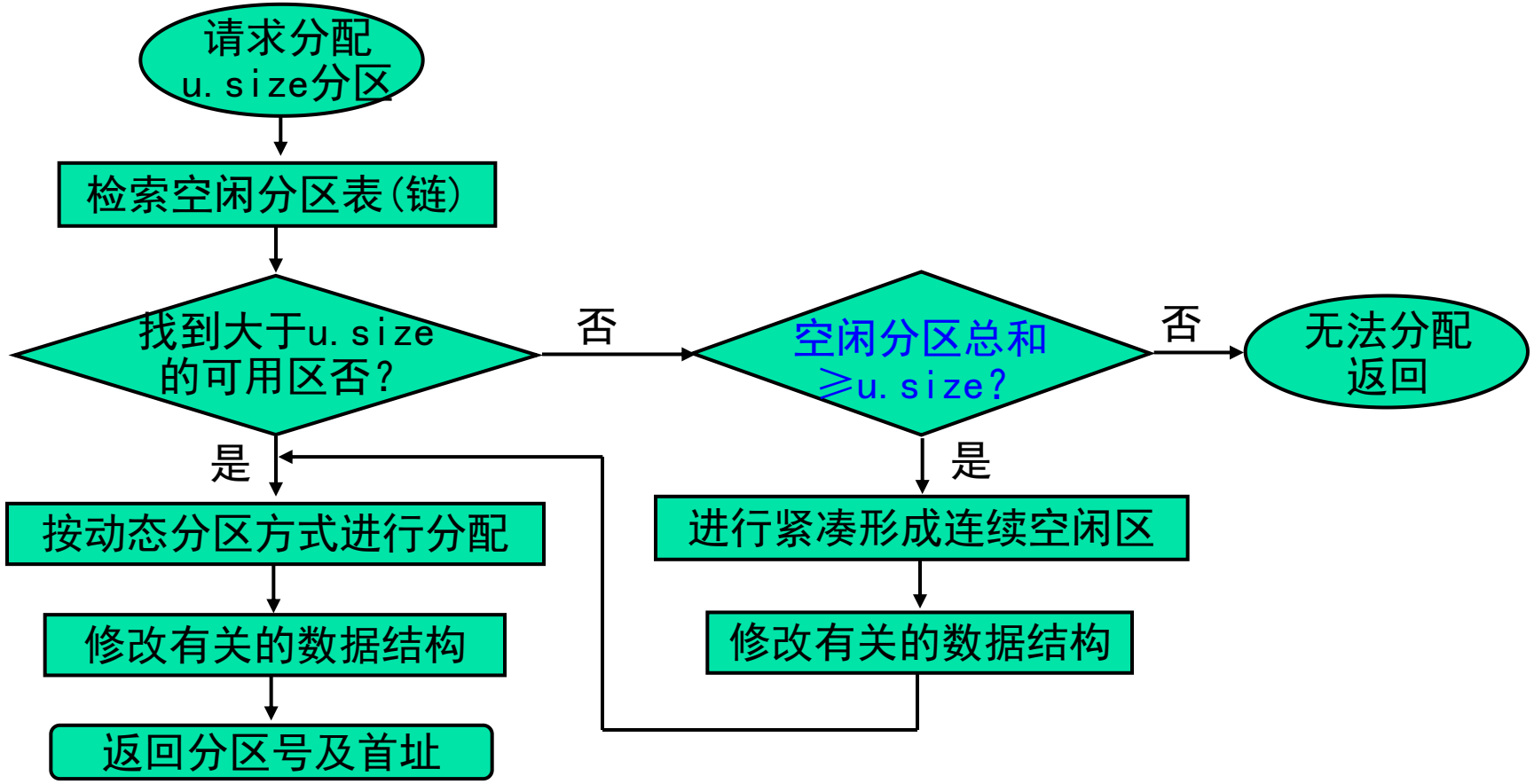


图5-6 动态分区分配算法流程图

5.3 对换（Swaping）

- **对换的引入**
  - 阻塞进程占用大量内存
  - 外存中的等待作业因无内存不能调入运行

  } 资源浪费

- **对换技术**
  - 也称交换技术，把内存中暂时不能运行的进程或进程所需要的程序和数据，调出到外存上，以便腾出足够的内存空间，再把已具有运行条件的进程或进程所需要的程序和数据，调入内存。

5.3 对换（Swaping）

■ 对换空间的管理

整体对换
（进程对换）

- 对换以整个进程为单位
- 广泛应用于分时系统中

交换调度

部分对换

- 对换以“页”为单位——页面对换
- 对换以“段”为单位——分段对换

虚存

本节介绍进程对换，部分对换将在虚拟存储器中介绍。

要实现进程对换，需实现三方面功能：

- 对换空间的管理
- 进程的换出
- 进程的换入

5.3 对换（Swaping）

对换空间的管理

- 把外存分为：文件区和对换区

文件区

- 存放文件
- 管理主要目标：提高文件存储空间利用率
- 为此，采用离散分配方式

对换区

- 存放换出的进程
- 管理主要目标：提高进程换入和换出的速度
- 为此，采用连续分配方式

- 对换区空闲盘块的管理：
  - 空闲分区表（盘块号、长度）
  - 对换区空闲盘块分配算法

} 与内存连续分配方法类似

5.4 基本分页存储管理

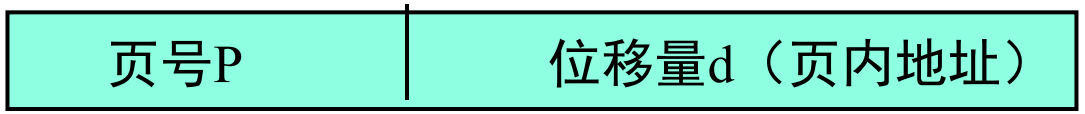
- **引入：**
  - 离散内存分配方式：提高内存空间利用率；支持程序动态增长
- **基本思想**
  - 将逻辑地址空间分成若干大小相等的片，称为页面或页(page)。页号从0开始。
  - 内存空间分成与页大小相同的若干存储块，称为块或页框(frame)。也从0开始编号。
  - 为进程分配内存时，以块为单位将进程中的若干页分别装入到多个可以不相邻的物理页中。
  - 逻辑地址可转化成 “页面号+页内偏移”
  - 建立页号和物理块号的对应关系（页表），并借助页表完成地址变换（重定位）。

5.4 基本分页存储管理

- **思考：几个问题的处理？**
  - 重定位的计算？
  - 需要保存哪些信息，对应的数据结构？
  - 分配和回收流程？

5.4.1 基本分页存储管理-地址结构

- 地址结构
  - 分页的逻辑地址可分解为“页号+页内偏移地址”表示，结构如下：

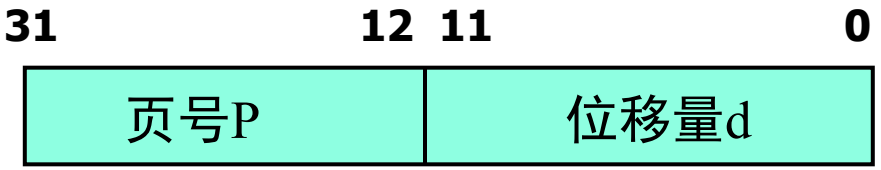


  - 设逻辑地址为A，页面大小为L，则页号P和页内地址d可按下式求得：

$$P=int(A/L)$$

$$d=A \bmod L$$

  - 如：设某逻辑地址空间地址长度为32位，每页大小为4KB，则地址空间大小就是2^{32}B，地址空间最多2^{20}个页。则地址结构如下，其中地址0–11位就可以标识了页内地址，12–31位标识了页号。



5.4.1 基本分页存储管理-地址结构

- 地址结构
  - 举例：设页面大小为4KB，逻辑地址为5F86H及7800，分别求它们的页号和页内偏移。
    - 1）页号P=INT(5F86H/4096)=5

      页内偏移=5F86H%4096=F86H
    - 2）页号P=INT(7800/4096)=1

      页内偏移=7800%4096=3704

5.4.2 基本分页存储管理-页表

- 页表
  - 系统为每个进程建立一张页表，记录了相应页在内存中对应的物理块号，实现从页号到物理块号的地址映射。

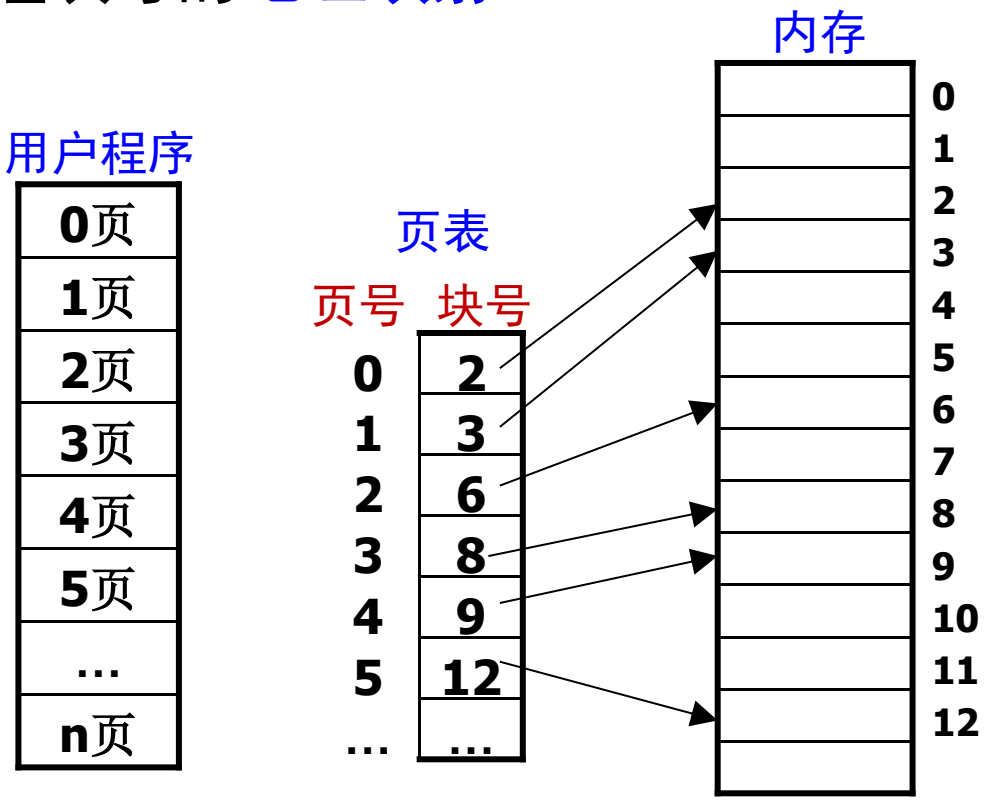


图5-8 页表

5.4.2 基本分页存储管理-地址变换

- 基本的地址变换机构
  - 页表驻留在内存中，设置一个页表寄存器存放页表在内存中的始址。地址转换的原理如图所示。

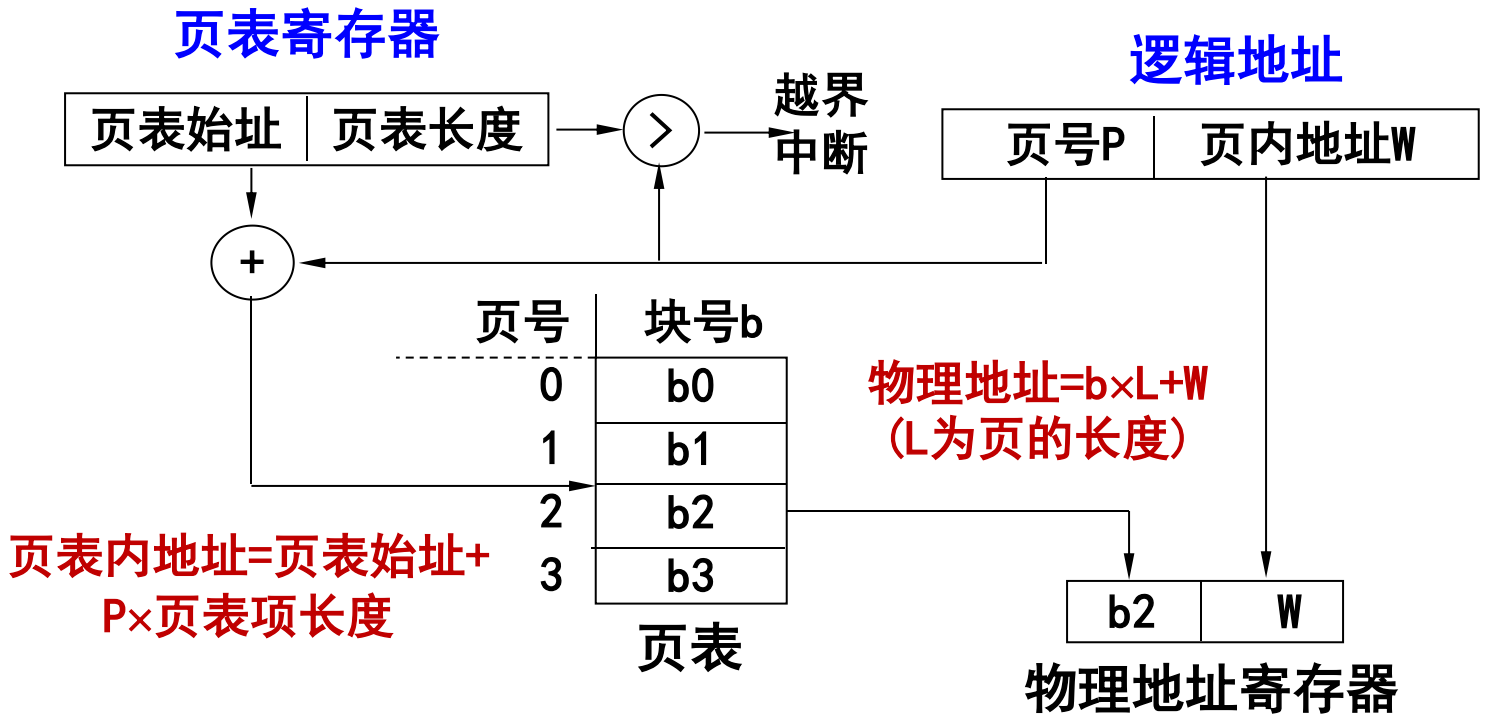


图5-9　分页系统的基本地址变换机构

5.4.3 基本分页存储管理-地址变换

- 举例
  - 在采用页式存储管理的系统中，某作业的逻辑地址空间为4页（每页4096字节），且已知该作业的页表如下表。试求出逻辑地址14688所对应的物理地址。

页　表

页号	内存块号
0	2
1	4
2	7
3	9

解：页号P=INT(14688/4096)=3

页内偏移d=14688%4096=2400(960H)

由页号3查页表，得块号为9

物理地址=9×4096+2400=39264(9960H)

对于十六进制，由块号9与页内偏移960可“合成”物理地址9960H。

问：若左边页表中的9改为30，则对应的物理地址是多少？

5.4.3 基本分页存储管理-地址变换

- 练习
  - 1、设有一页式存储管理系统，向用户提供的逻辑地址空间最大为16页，每页2048B，内存总共有8个存储块。试问逻辑地址至少应为多少位？内存空间有多大？

  - 2、在一分页存储管理系统中，逻辑地址长度为24位，页面大小为4096B，现有一逻辑地址为2FA6H，且第0、1、2页依次存放在物理块10、12、14中，问相应的物理地址为多少？

5.4.3 基本分页存储管理-地址变换

- 具有快表的地址变换机构
  - 由于页表存放在内存中的，因此CPU每存取一个数据，要访问2次内存。
    - 第1次：是访问内存中的页表，从中找到指定页的物理块号，以形成物理地址。
    - 第2次：从物理地址中获取或写入数据，因此计算机处理速度降低近1/2。
  - 快表：为提高地址转换速度，在MMU(Memory Management Unit)中增设一个具有并行查寻能力的特殊高速缓冲寄存器，称为联想寄存器(Associative Memory)，或称快表，用于存放当前访问的那些页表项。

5.4.3 基本分页存储管理-地址变换

- 具有快表的地址变换机构

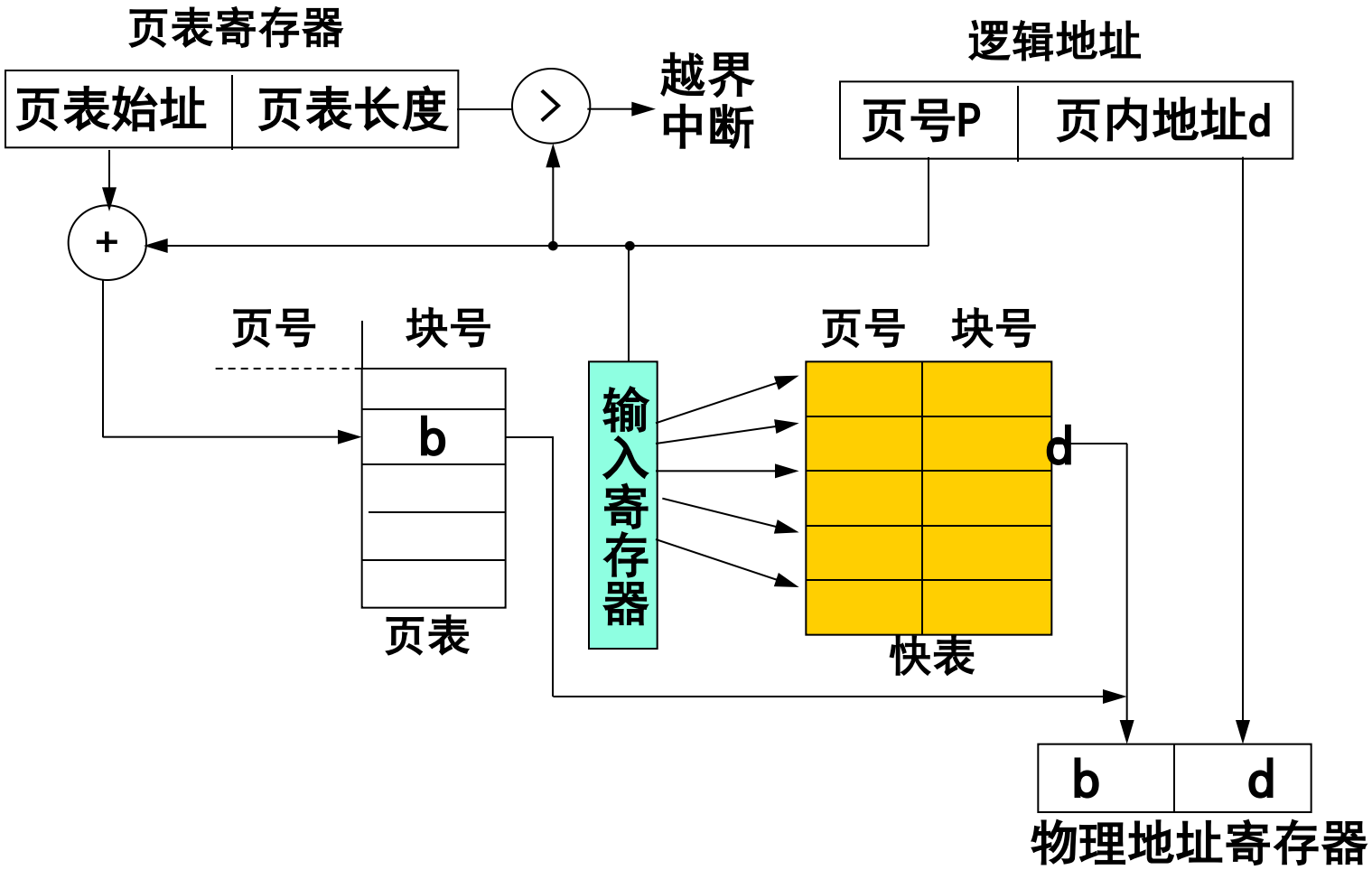


图5-10　具有快表的地址转换机构

5.4.4 基本分页存储管理-多级页表

- 引入：
  - 考虑具有32位逻辑地址空间的分页系统：若页面大小为4KB（即2^{12}B），则每个进程页表中的页表项可达1兆(1024K=2^{20})个，若每个页表项占用4个字节，故每个进程仅页表就要占用1024K*4B=4MB的内存空间，而且还要求是连续的——不现实或受限。
  - 解决办法之一：采用离散存储方式存储一级页表
- 两级页表
  - 将页表分页
  - 将页表的各个页面离散地存放在不同的物理块中
  - 为离散存储的页表再建立一张页表，称为外层（外部）页表。

5.4.4 基本分页存储管理-多级页表

- 两级页表结构

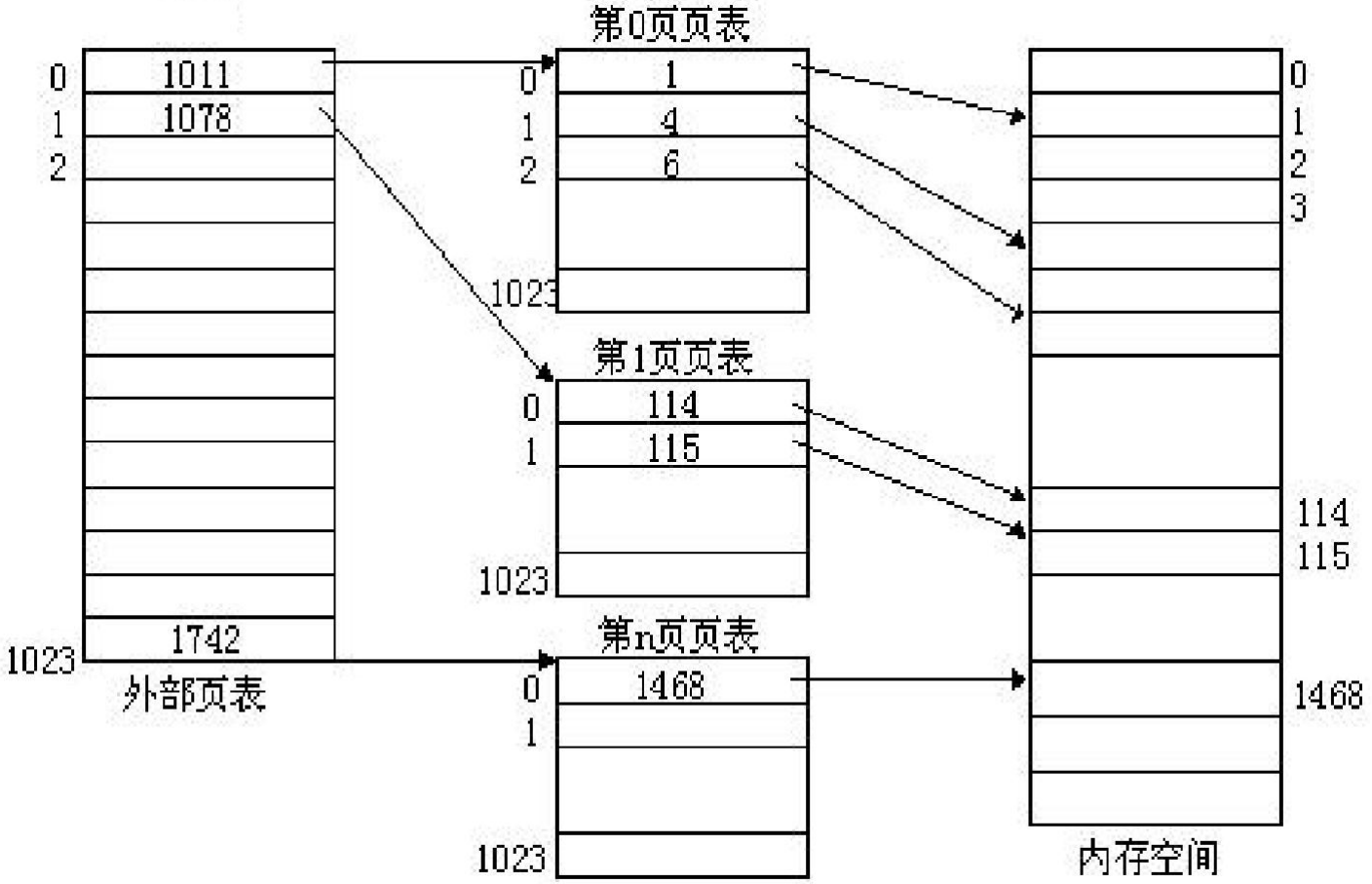
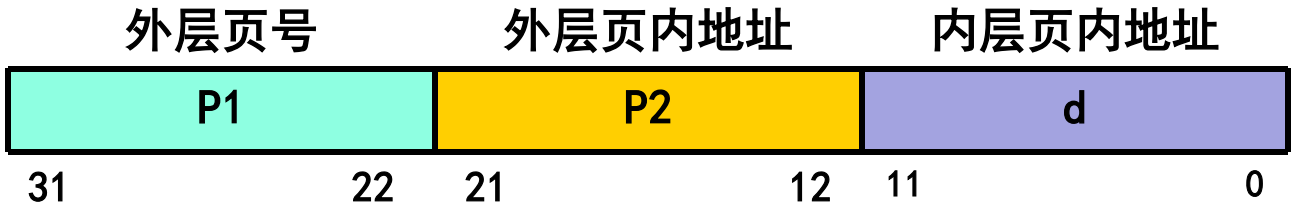


图5-11　两级页表结构

5.4.4 基本分页存储管理-多级页表

- 两级页表逻辑地址结构
  - 以32位逻辑地址空间为例，则逻辑地址空间大小为2^{32}字节（2^{32}B），假设页面大小为4KB=2^{12}B时，若采用一级页表，则页表共有$2^{32}/2^{12}=2^{20}$个页表项，即20位的页号，页内地址d是12位。
  - 采用两级页表结构时，对页表再分页（内层页表和外层页表）
  - 假设每个页表项占4个字节（4B），则内层页表的每个页中包含4KB/4B=2^{10}个页表项，因此外层页表中的页内地址P2是10位
  - 则外层页表最多需要$2^{32}/2^{12}/2^{10}=2^{10}$个页存放内层页表，即外部页表中页号P1也是10位，此时逻辑地址结构如下：



  - 思考：如果每个页表项占2个字节，逻辑地址结构情况？

5.4.4 基本分页存储管理-多级页表

- 两级页表地址变换
  - 为地址变换方便，需设置一个外层页表寄存器。两级页表地址变换机构如图所示(图中假设页的大小为4KB)

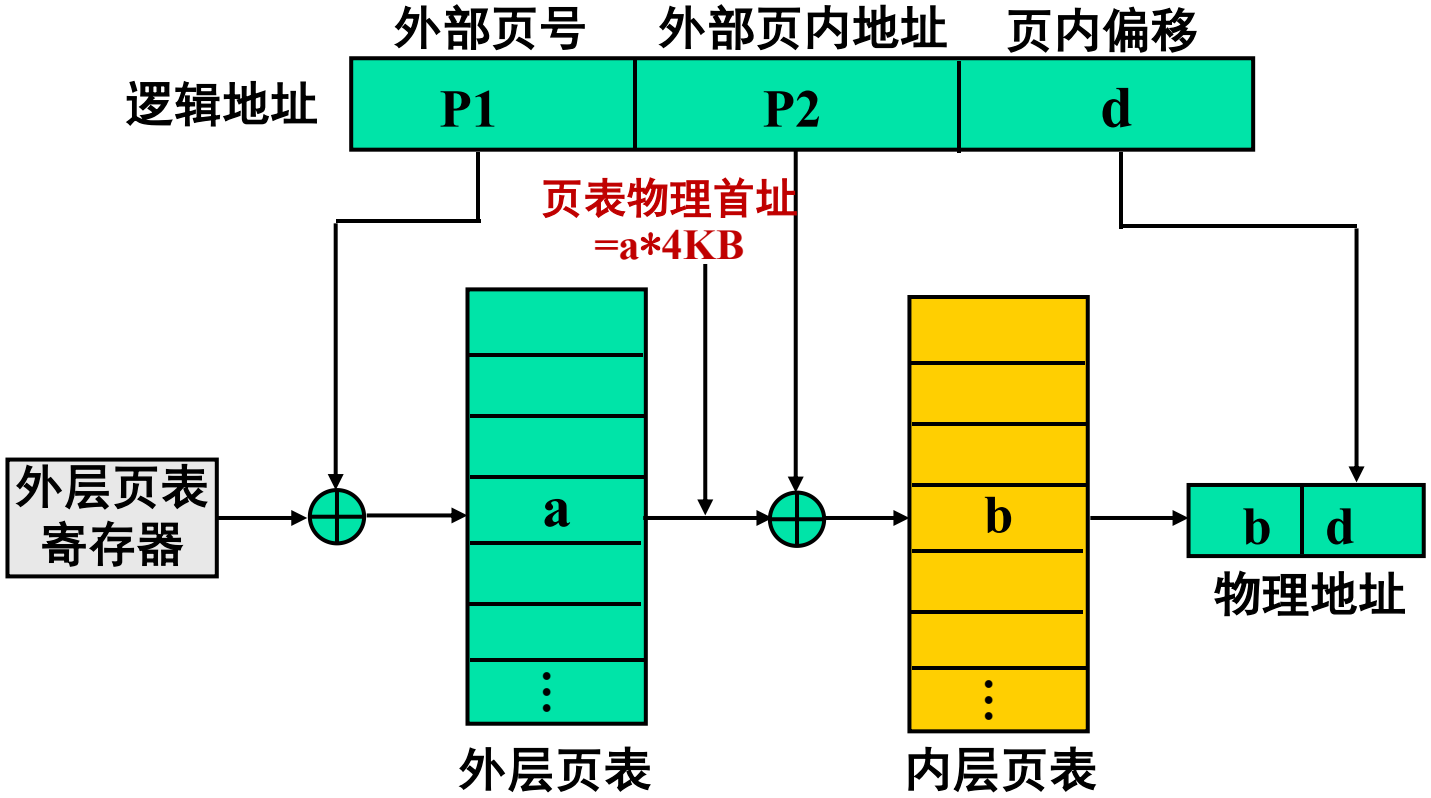


图5-12　两级页表地址变换

5.4.4 基本分页存储管理-多级页表

- 多级页表
  - 例：设逻辑地址空间64位地址，设页大小为4KB=2^{12}（12位），则块号占64−12=52位（每个块号用8字节存储）。若每个块号（页表项）需要8个字节，则每块可存储512=2^9个块号（故页表索引号占9位）。则外层页号占43位（有2^{43}个页表），2^{43}个表项，因每个表项8（即2^3）个字节，所以存放外层页表需要2^{46}个字节（64TB），大得吓人 ——两级页表已经不行了

即使用3级页表也不行



外层页表有2^{34}个表项，占2^{37}字节，128GB。

多级页表

5.4.5 基本分页存储管理-反置页表

- 反置页表（Inverted Page Table）的引入
  - 反置页表是为每个物理块设置一个页表项，并将它们按照物理块的编号排序，其中的内容是页号和所隶属的进程标识号。
- 反置页表地址变换过程
  - step1:根据进程标识号和页号，检索反置页表。
  - step2:如果检索到与之匹配的页表项，则该页表项的序号便是该页所在的物理块号，用该块号与业内地址一起构成物理地址。
  - step3:若找不到该表项，则表示该页未调入内存。

5.4.5 基本分页存储管理-反置页表

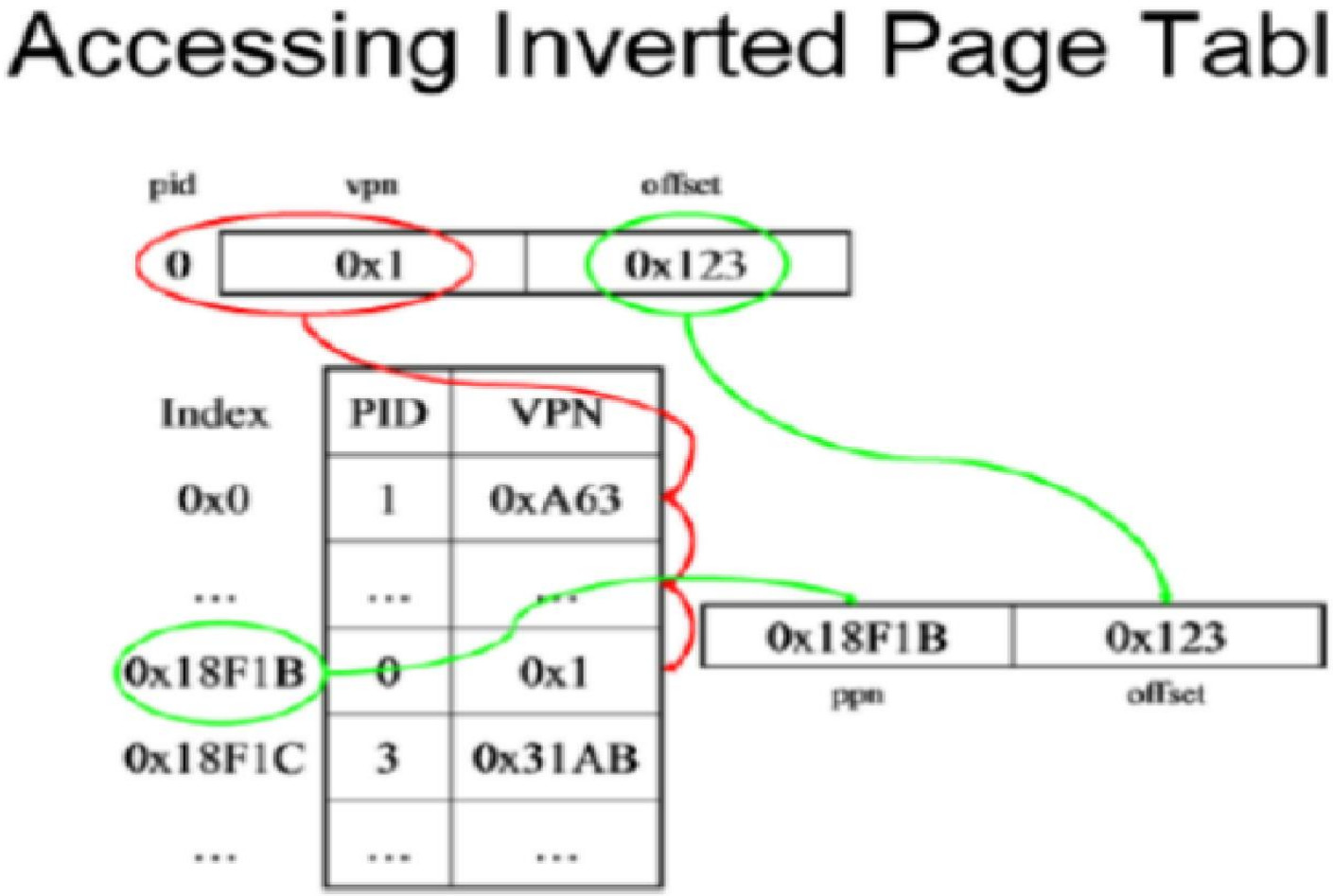


图5-13　反置页表地址变换

5.4.5 基本分页存储管理-反置页表

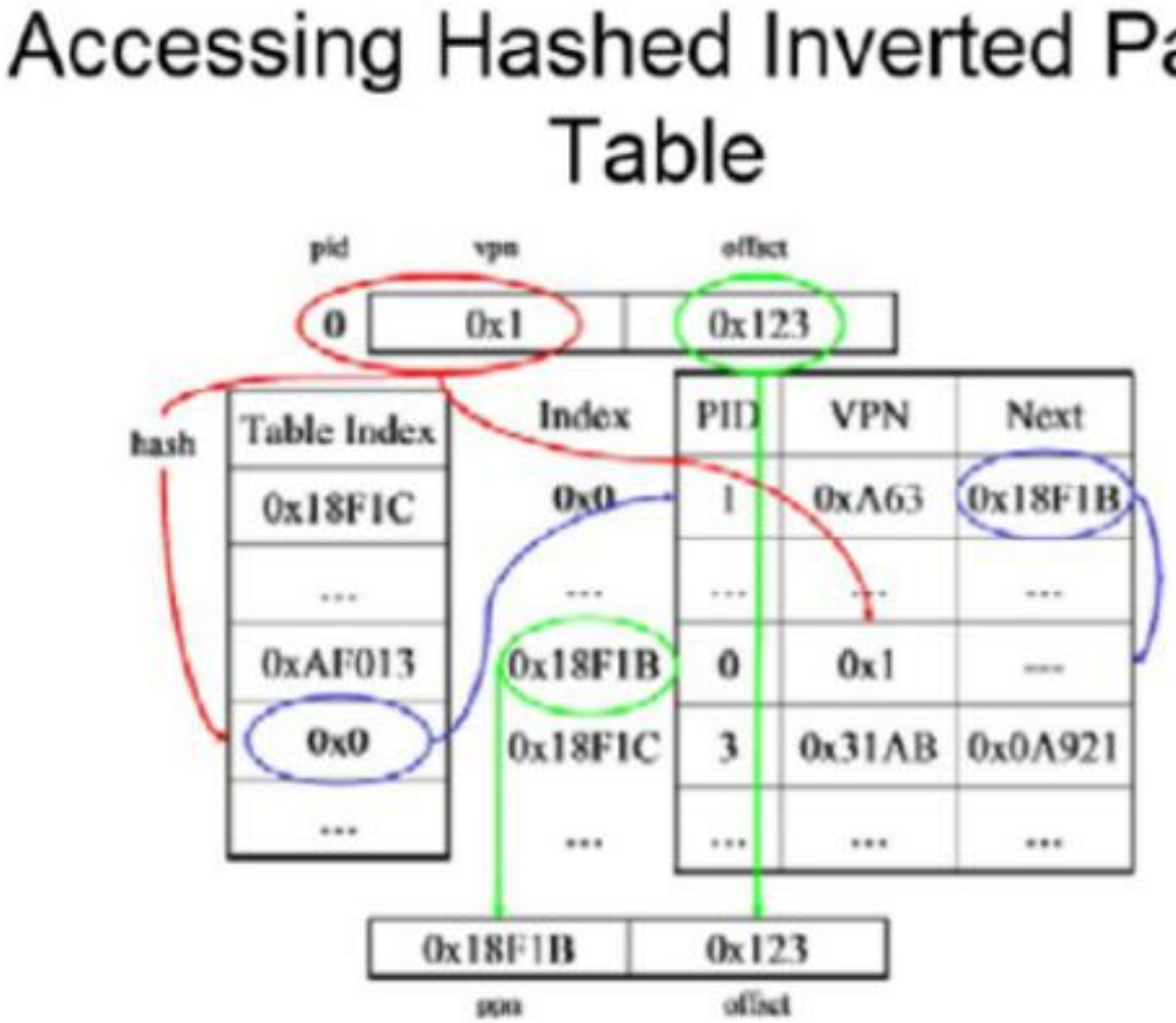


图5-13 Hash检索反置页表地址变换

5.4.4 基本分页存储管理

- 练习：
  - 1、已知某系统页面长4KB，页表项4B，采用多级页表映射64位虚地址空间。若限定最高层页表占1页，问它可以采用几级页表？
  - 2、设某系统页面大小为4KB，逻辑地址为40B，内存大小为128MB，则反向页表的长度(表项数)是多少？
  - 3、为实现分页存储管理，需要哪些硬件支持？

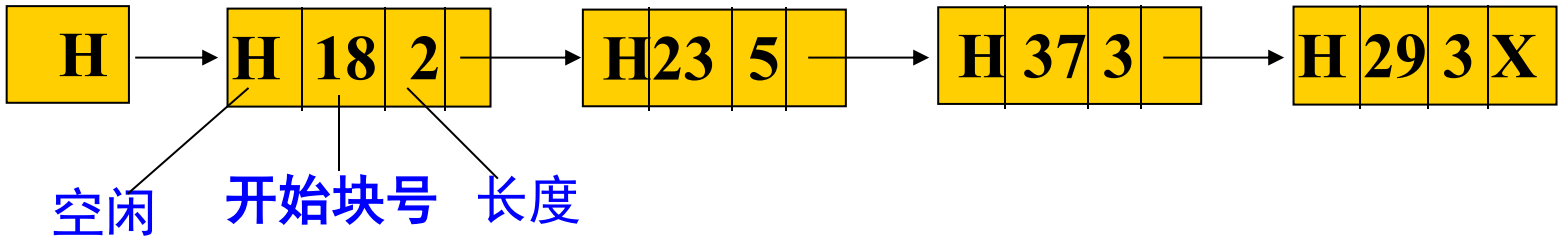
5.4.5 分页存储内存空间的管理

- 数据结构
  - 位示图

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	1	0	0	0	1	1	1	0	0	1	0	1	1	1	0
1	0	0	0	1	1	1	1	1	1	0	0	0	0	1	1	1
2	1	1	1	0	0	0	1	1	1	1	1	1	0	0	0	0
3																
…																
15																

位示图

  - 空闲块表（链）方法

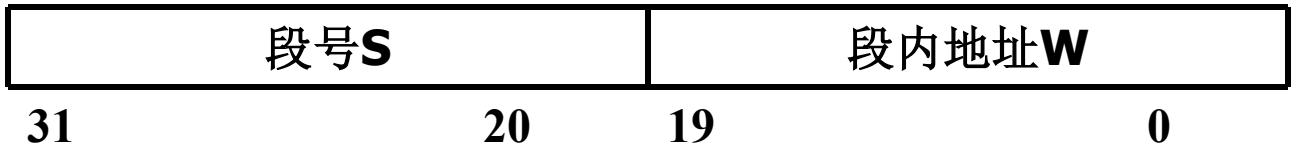


5.5 基本分段存储管理

- 分段存储管理方式的引入：
  - 方便编程
  - 信息共享
  - 信息保护
  - 动态增长
  - 动态链接

5.5 基本分段存储管理

- 基本分段存储管理思想
  - 作业地址空间划分成若干段：主程序段，数据段，堆栈段等。
  - 每个段都有名字，常用段号代替段名（段号从0开始）。
  - 每个段内都从0开始编址，逻辑地址由“段号+段内偏移地址”组成。



  - 每个段占一个连续内存空间，各个段之间不要求连续。
  - 设置一个段表地址寄存器，保存当前执行进程段表的起始地址和长度
  - 思考：几个问题的处理？

5.5 基本分段存储管理

- 段表

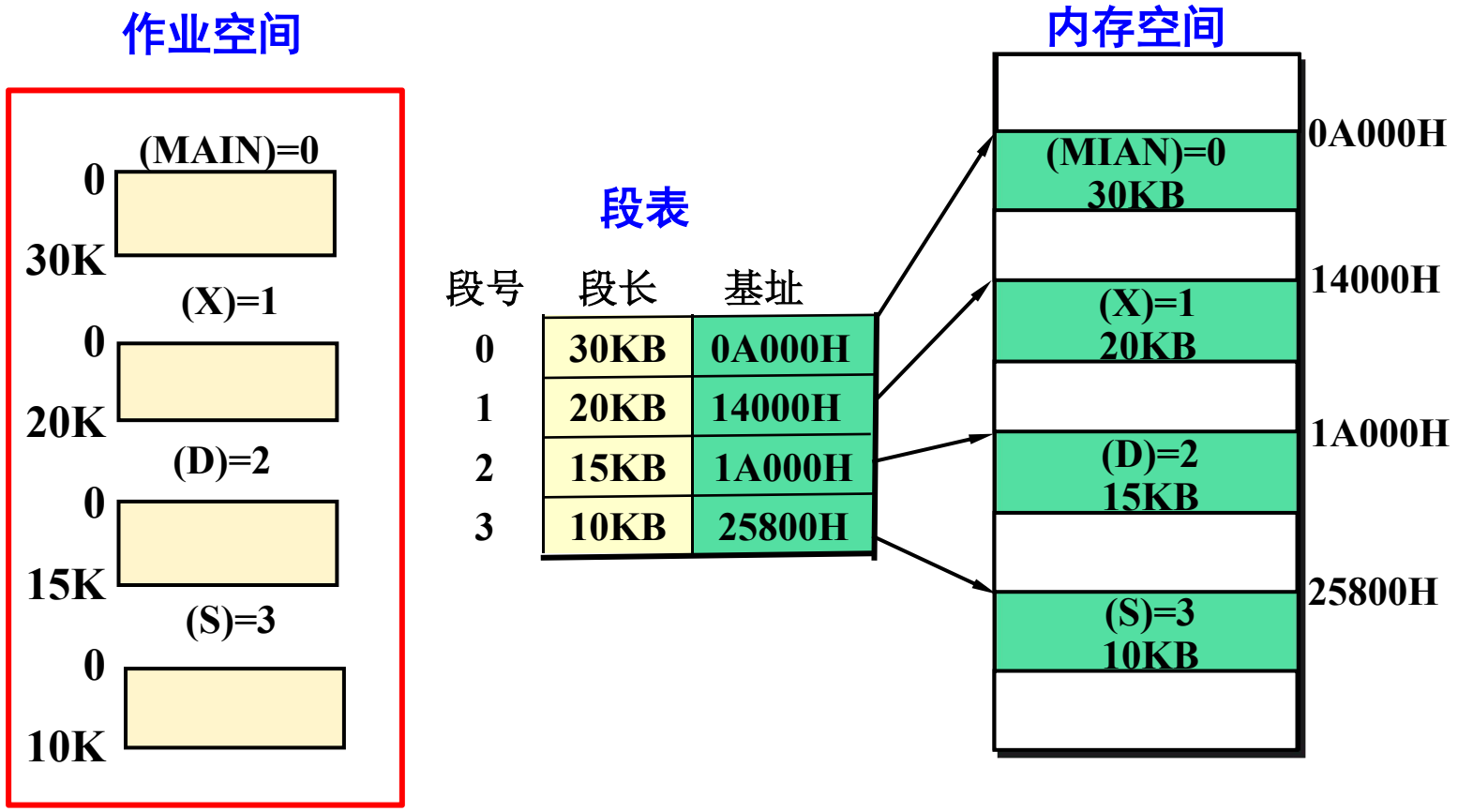


图5-13　利用段表实现地址映射

5.5 基本分段存储管理

- 基本分段地址变换

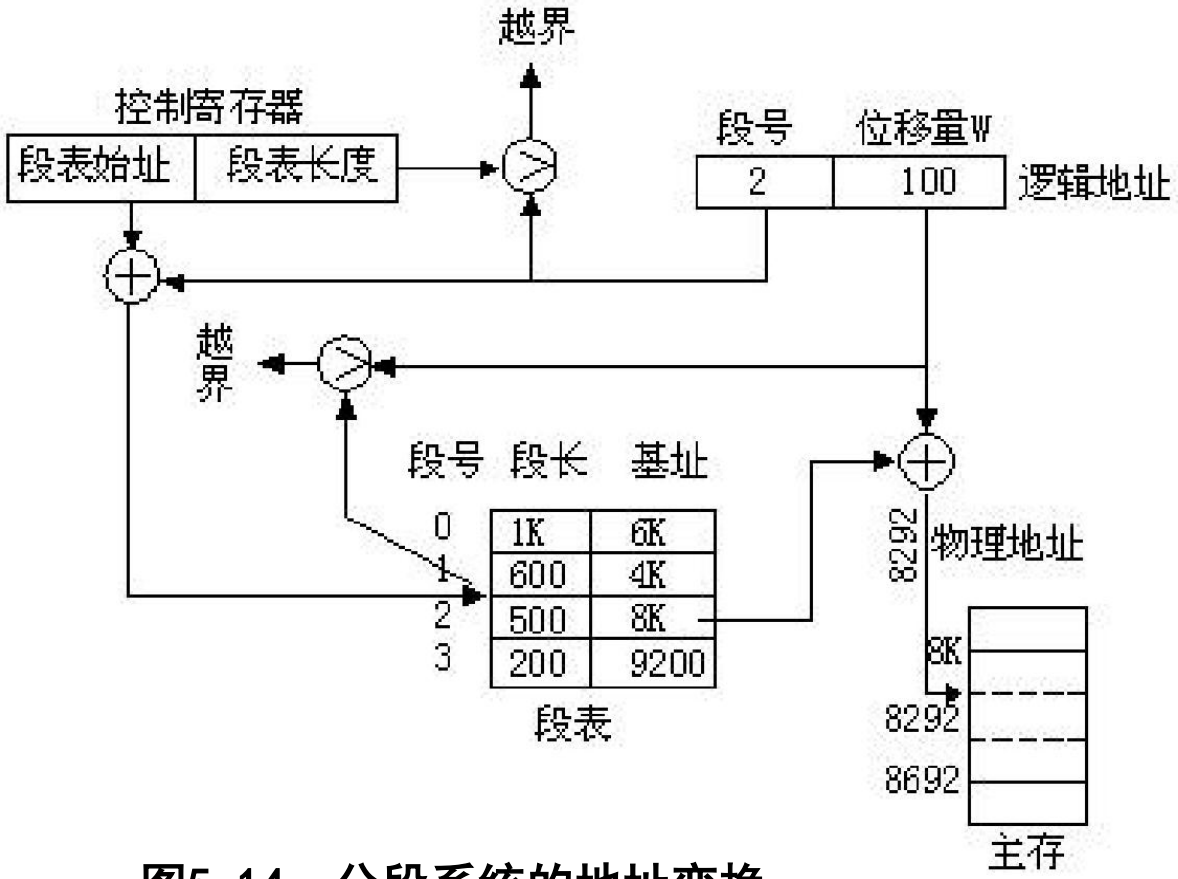


图5-14　分段系统的地址变换

- **思考：段式管理下每访问一个数据，要访问几次内存？**

5.5 基本分段存储管理

- 信息共享

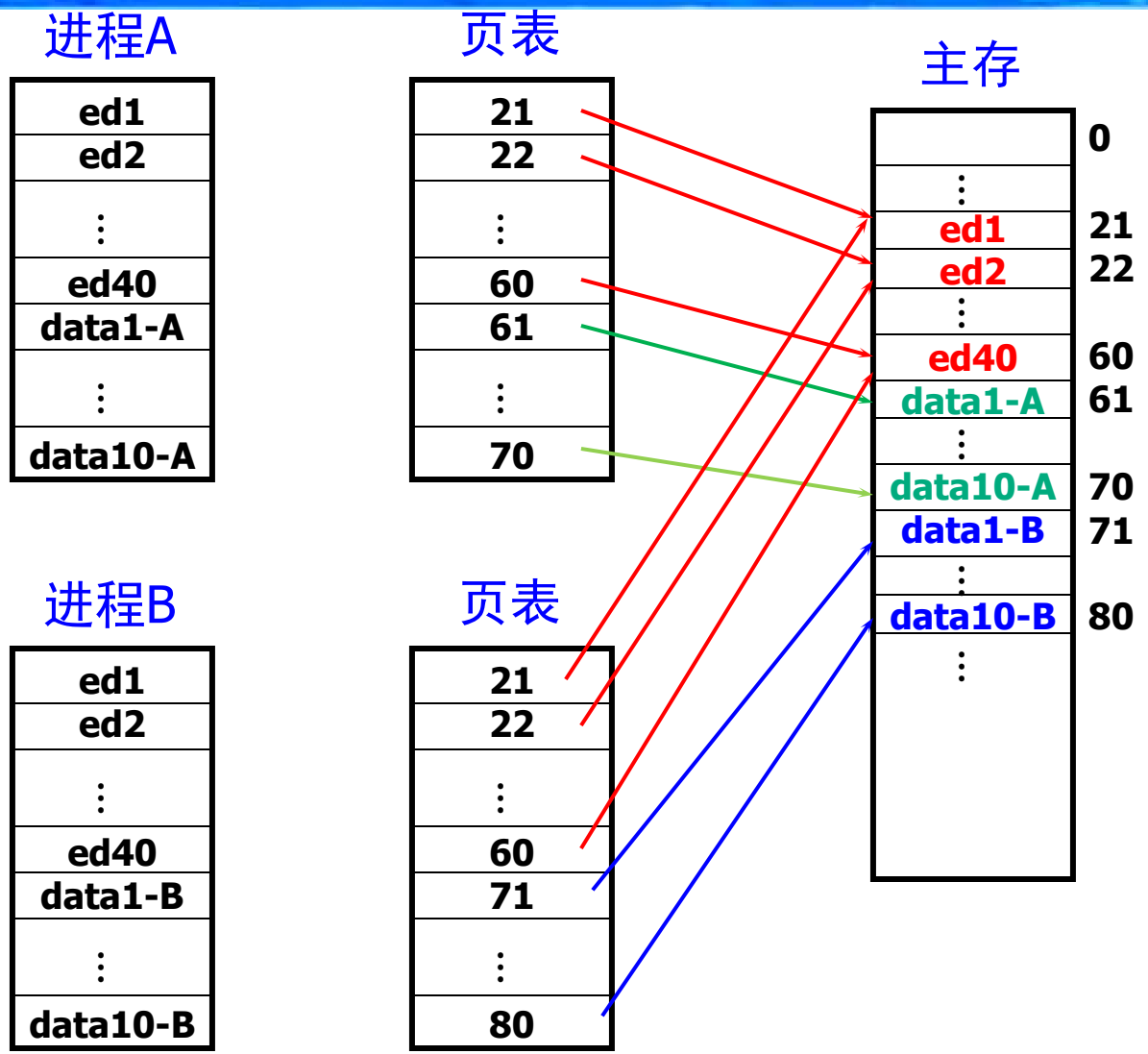


图5-15　分页系统共享

5.5 基本分段存储管理

- 信息共享

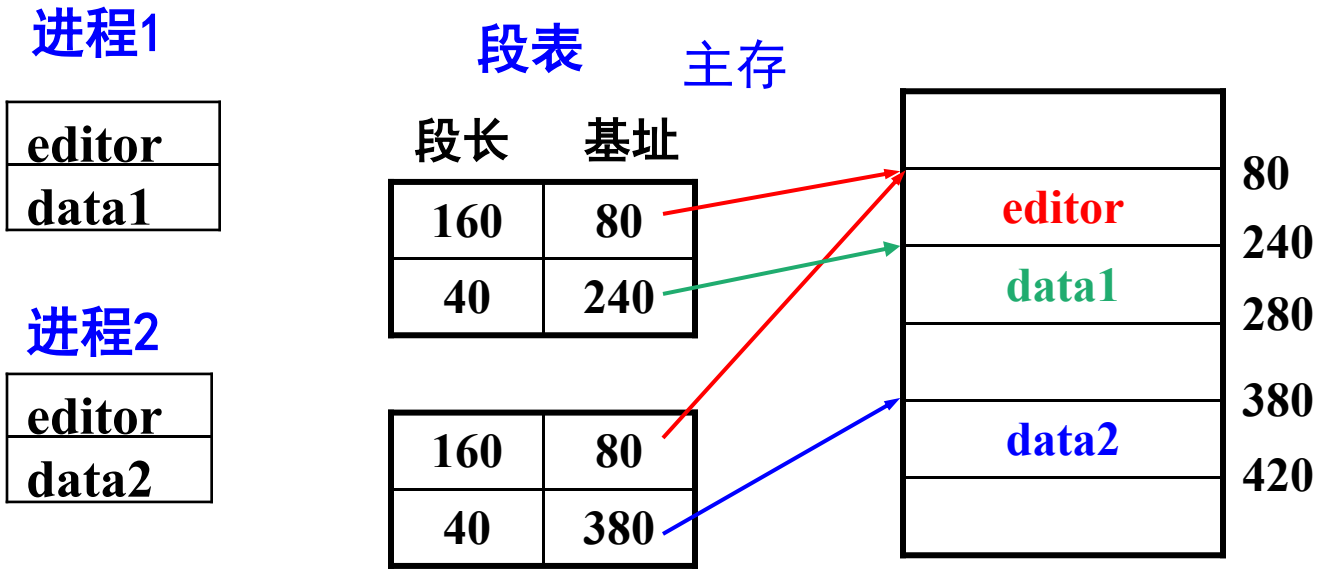


图5-16　分段系统共享

5.6 段页式存储管理

- 引入
  - 取长补短（页式：空间利用率高；段式：满足用户需求）
- 基本思想
  - 内存物理地址空间等分成物理块（页框），并从0连续编号
  - 作业的逻辑地址先分成若干段，各段按页框大小等分成页，并从0连续编号
  - 作业各段连续的页，可以分配不连续的页框
  - 逻辑地址由3部分组成：段号、段内页号、页内地址

段号S	段内页号P	页内地址W

  - 系统为每个作业设置一个段表，每个分段再设置一个页表，利用段表和页表实现地址映射
  - 设置一个段表地址寄存器，保存当前执行进程段表的起始地址和段表的长度

5.6 段页式存储管理

- 基本思想

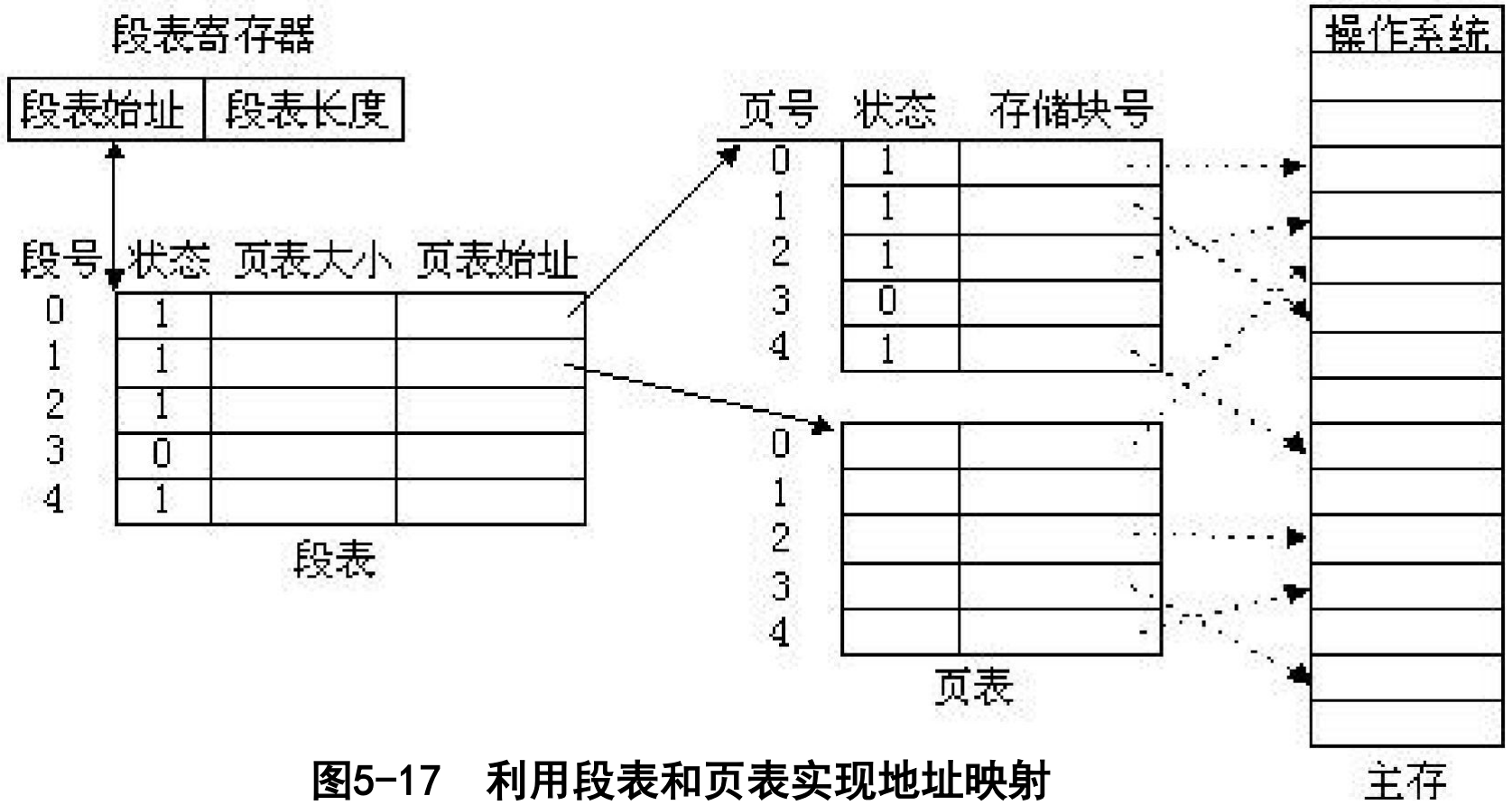


图5-17　利用段表和页表实现地址映射

5.6 段页式存储管理

- 地址变换过程

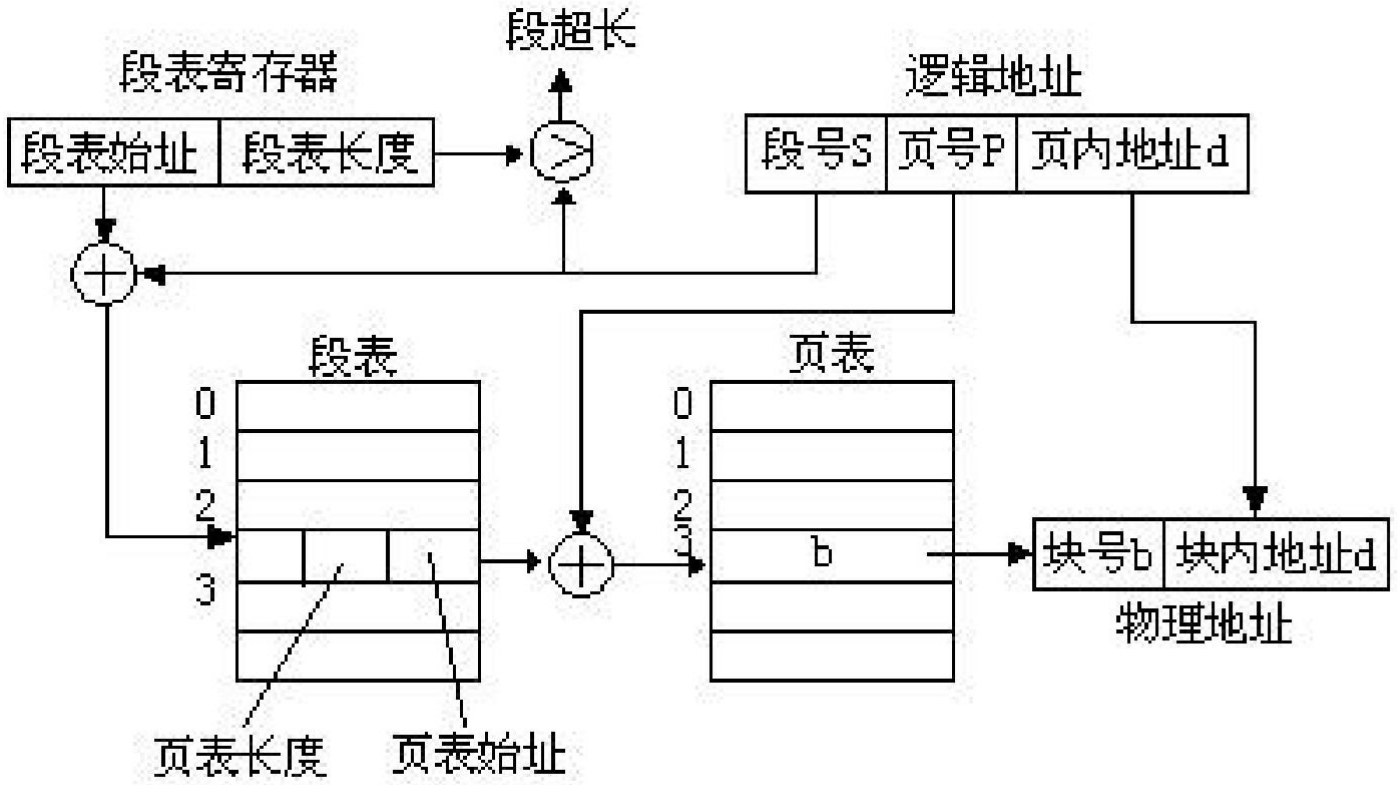
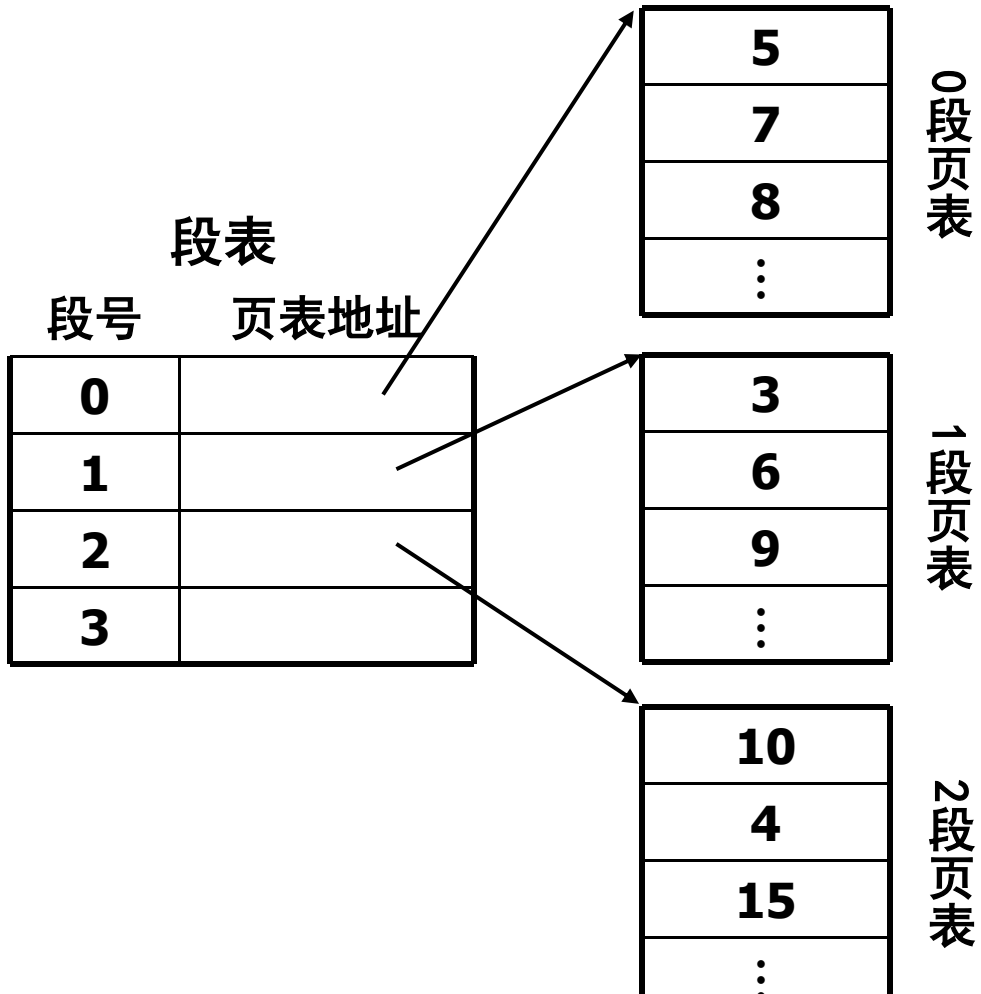


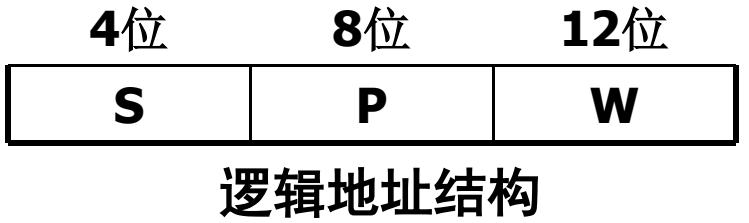
图5-18　段页式系统地址变换

- **思考：段页式管理下每访问一个数据，要访问几次内存？**

5.6 段页式存储管理

- 练习：
  - 某系统采用段页式存储管理，其逻辑地址结构和某作业的段表、页表结构如下图所示。请计算该作业中逻辑地址10248AH对应的物理地址





5.6 段页式存储管理

- 碎片（Fragmentation）
  - 碎片是系统中未被使用的空闲内存，但由于其大小或位置不连续，无法满足新的内存分配请求，导致存储资源浪费。
    - 内部碎片：已分配的内存块大于实际需求，多余部分无法被其他进程使用
    - 外部碎片：系统中存在多个不连续的小空闲内存块，但因无法或难以合并而无法有效利用
  - 思考：前面各种内存分配方式下，会产生哪些类型的碎片？
    - 连续分配：单一连续分配/静态分区分配/动态分区分配
    - 离散分配：页式/段式/段页式

5.6 段页式存储管理

- 有效访问时间
  - 有效访问时间(Effective Access Time,EAT)是指从进程发出指定逻辑地址的访问请求，经过地址变换，到在内存中找到对应的实际物理地址单元并取出数据，所需要花费的总时间，称为内存的有效访问时间。
- 基本页式存储管理EAT
  - 无快表
    - 假设一次访问内存的时间为t，则：（EAT=？）

      EAT=t+t=2t
  - 有快表
    - 假设一次访问快表的时间为λ，快表命中率a，一次访问内存的时间为t，则（EAT=？）

      EAT=a*λ+(1-a)*(λ+t)+t或EAT=a*(λ+t)+(1-a)*(λ+t+t)

5.7 虚拟存储器的基本概念

- 引入
  - 常规存储器管理方式的特征
    - 一次性
    - 驻留性
  - 带来的问题
    - 当作业要求的内存空间超过内存总容量时，无法装入内存运行
    - 程序运行中不用或暂时不用的程序（数据）占据大量内存空间

5.7 虚拟存储器的基本概念

- 局部性原理
  - 程序在执行时将呈现局部性规律，即在一较短的时间内，程序的执行仅局限于某个部分（如顺序执行，过程调用，循环，数组访问等）；相应地，它所访问的存储空间也局限于某个区域。
  - 时间局部性——某指令一旦执行，则不久后该指令可能再次被执行。（循环）
  - 空间局部性——程序一旦访问了某个存储单元，不久后，附近的存储单元也将被访问。（顺序）

5.7 虚拟存储器的基本概念

- 虚拟存储器的定义
  - 虚拟存储器是指具有请求调入功能和置换功能，能从逻辑上对内存容量加以扩充的一种存储系统。
- 基本思想：

部分装入	请求调页	页面置换

- 虚拟存储器的特征
  - 多次性：一个作业被多次调入内存运行
  - 对换性：作业在运行过程中进行换进、换出
  - 虚拟性：从逻辑上扩充内存。

5.7 虚拟存储器的基本概念

- 虚拟存储器的实现
  - 请求分页（段）系统：是在分页（段）系统基础上，增加了请求调页（请求调段）功能和页面置换（段置换）功能所形成的页式（段式）虚拟存储系统。
- 思考：需要补充哪些工作？
  - 页表（段表）：请求分页的页表/请求分段的段表
  - 缺页中断（缺段中断）处理
  - 地址变换过程
  - 页面置换（段置换）算法

5.8 请求分页存储管理

- 页表
  - 每个页表表项如下所示：

页号	访问位A	修改位M	状态位P	物理块号	外存地址

  - 其中：
    - 状态位P：指示该页是否已调入内存
    - 访问位A：记录本页在一段时间内被访问的次数
    - 修改位M：表示该页被调入内存后是否被修改过
    - 外存地址：指出该页在外存的地址，通常是磁盘物理块号

5.8 请求分页存储管理

- 地址变换过程

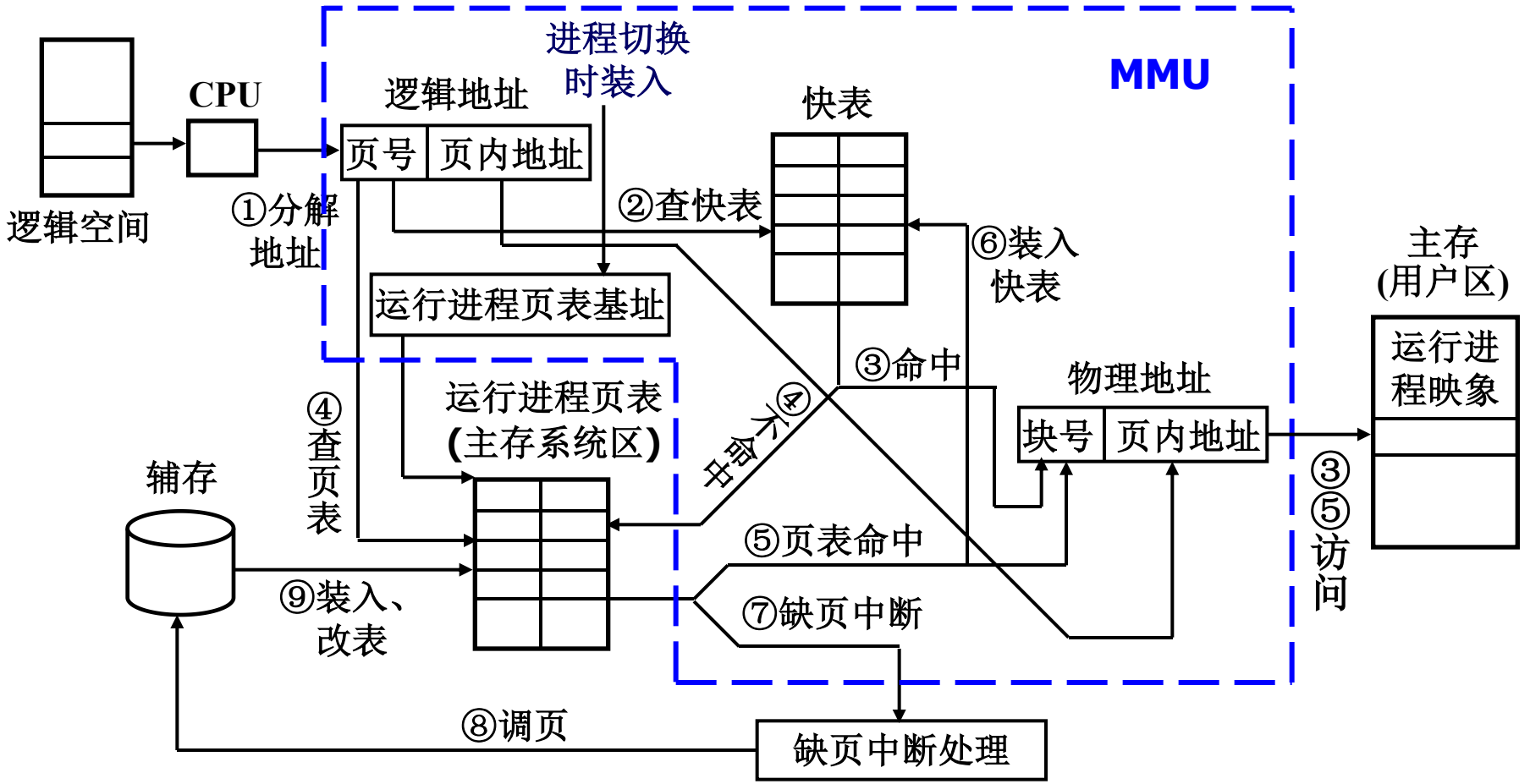


图5-19　请求调页系统地址变换过程

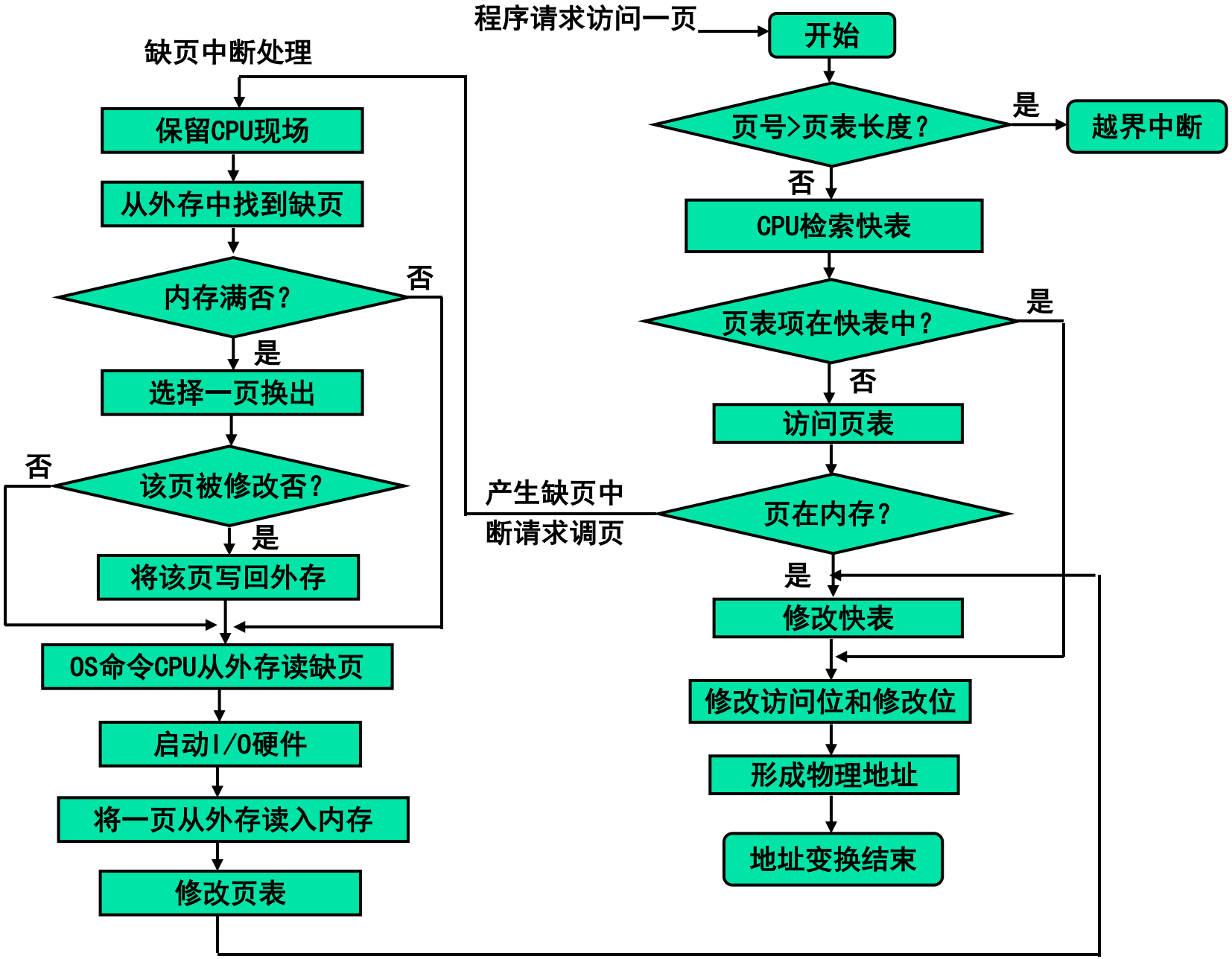


图5-20　请求调页系统地址变换和缺页中断

5.8 请求分页存储管理

- 软件部分的策略
  - 内存分配策略
  - 调页策略
  - 页面置换算法

5.8 请求分页存储管理

- 内存分配策略
  - 最小物理块数的确定
    - 取决于指令的格式、功能和寻址方式
  - 物理块的分配策略
    - 固定分配局部置换
    - 可变分配全局置换
    - 可变分配局部置换
  - 物理块分配算法
    - 平均分配算法
    - 按比例分配算法
    - 考虑优先权的分配算法

5.8 请求分页存储管理

- 调页策略
  - 何时调入页面
    - 预调策略(预测准确率、工作集等)
    - 请求调页页策略(系统开销)
  - 从何处调入页面
    - 系统有足够的对换区——全部从交换区调入
    - 系统缺少交换区——全部从文件区调入
    - 凡未运行的页面，从文件区调入；曾经运行过又被调出的页面，从交换区调入（UNIX）
  - 页面调入
    - 有空闲内存物理块，页面直接调入
    - 没有，则页面置换，页面换出之前考察是否需要回写回磁盘

5.8 请求分页存储管理

- 缺页率
  - 进程的逻辑空间为n页，系统为其分配的内存物理块数为m(m≤n)。在进程的运行过程中，访问页面成功的次数为S，访问页面失败的次数为F，则该进程总的页面访问次数为A，那么该进程在其运行过程中的缺页率f为：

$$f = \frac{F}{A}$$

  - 思考：影响缺页率的因素？
    - 页面大小，物理块数、页面置换策略，程序本身的局部化程度等

5.9 页面置换算法

- **最佳置换算法（OPT）**
  - 思想：选择以后永不使用的或者是未来最长时间内不再使用的页面淘汰（一种理论上的算法）

【例】假定系统为某进程分配了3个物理块，并考虑以下的页面引用串：7，0，1，2，0，3，0，4，2，3，0，3，2，1，2，0，1，7，0，1（20个页面请求）

引用串	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
物理块	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
			1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
被置换的页				7		1		0			4			3				2		

图5-21 利用最佳页面置换算法的置换图

- 发生6次页面置换，缺页率（6+3）/20

5.9 页面置换算法

- **先进先出页面置换（FIFO）**
  - 思想：选择在内存中驻留时间最长的页面淘汰。
  - **数据结构支持**：设置一个指针，指向最老的页面

【例】假定系统为某进程分配了3个物理块，并考虑以下的页面引用串：7，0，1，2，0，3，0，4，2，3，0，3，2，1，2，0，1，7，0，1（20个页面请求）

引用串	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
物理块	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
		0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
			1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1
被置换的页				7			0	1	2	3	0	4			2	3			0	1	2

图5-22 利用FIFO页面置换算法的置换图

- 发生12次页面置换，缺页率（12+3）/20

5.9 页面置换算法

- **先进先出页面置换（FIFO）**
  - Belady现象：在采用FIFO算法时，有时会出现分配的物理页面数增加，缺页率反而提高的异常现象。
  - **【例】假定系统为某进程分配了3个物理块，并考虑以下的页面引用串：7，0，1，2，0，3，0，4，2，3，0，3，2，1，2，0，1，7，0，1（20个页面请求）**
  - **练习：可以自己比较下分配的物理块数分别为3和4的缺页率情况。**

5.9 页面置换算法

- 最近最久未使用置换算法（LRU）
  - 思想：选择最近最久未使用的页面进行淘汰。——用“最近的过去”作为“最近的将来”的近似。

【例】假定系统为某进程分配了3个物理块，并考虑以下的页面引用串：7，0，1，2，0，3，0，4，2，3，0，3，2，1，2，0，1，7，0，1（20个页面请求）

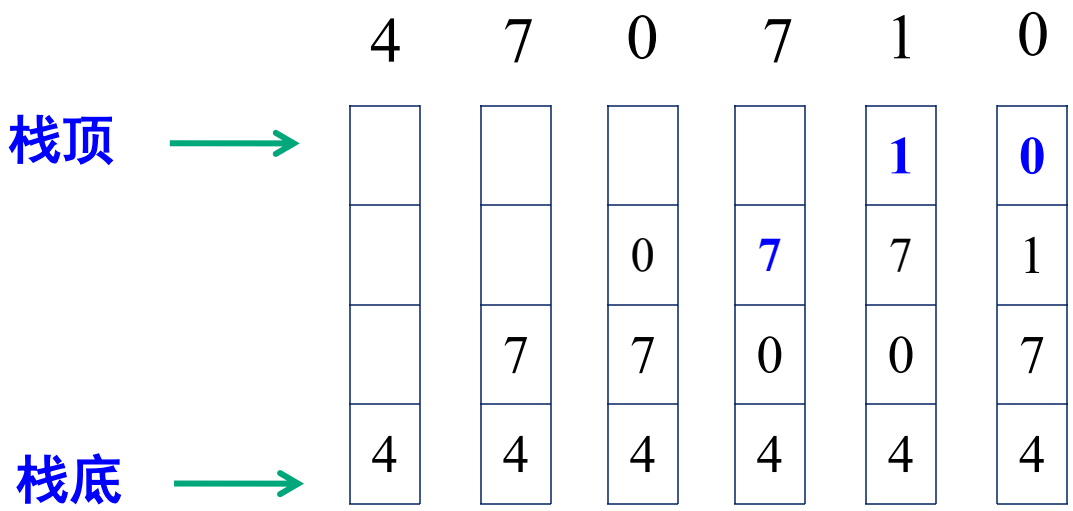
引用串	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
物理块	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
被置换的页				7		1		2	3	0	4			0		3		2		

图5-23　LRU页面置换算法的置换图

- 发生9次页面置换，缺页率（9+3）/20

5.9 页面置换算法

- LRU置换算法的实现
  - 寄存器
    - 为每个页设置一个多位寄存器，开始时，所有寄存器全部清0， $R=R_nR_{n-1}R_{n-2}...R_1$
    - 当页面被访问时，将它对应的寄存器的最左边位（最高位R_n）置1。每隔一定时间t，将所有页的寄存器右移一位。
    - 在发生缺页中断时，选择最小数值的寄存器对应页面淘汰
  - 栈



5.9 页面置换算法

- 最少使用页面置换算法（LFU）
  - 思想：选择最近使用频数最少的页面淘汰.
  - 实现：
    - 为每个页设置一个多位计数器，开始时，所有计数器全部清0。
    - 每次访问某个页时，就将它对应的计数器加1。隔一定时间t将所有页的计数器清0。
    - 在发生缺页中断时，可选择计数值最小的对应页面淘汰，并将所有计数器全部清0。

5.9 页面置换算法

- Clock页面置换算法
  - 思想
    - 每页设置一个访问位A
    - 在内存中的页面链接成一个循环队列，有一个指针指向最老的页面

- 当发生页面失效时，首先检查指针所指的页面，若它的A位是0就淘汰该页面，并把新页面插入这个位置，指针前移一个位置到C页；
- 若A位是1，就清除A位，并把指针前移一个位置；重复这个过程，直到找到一个A位是0的页面为止。

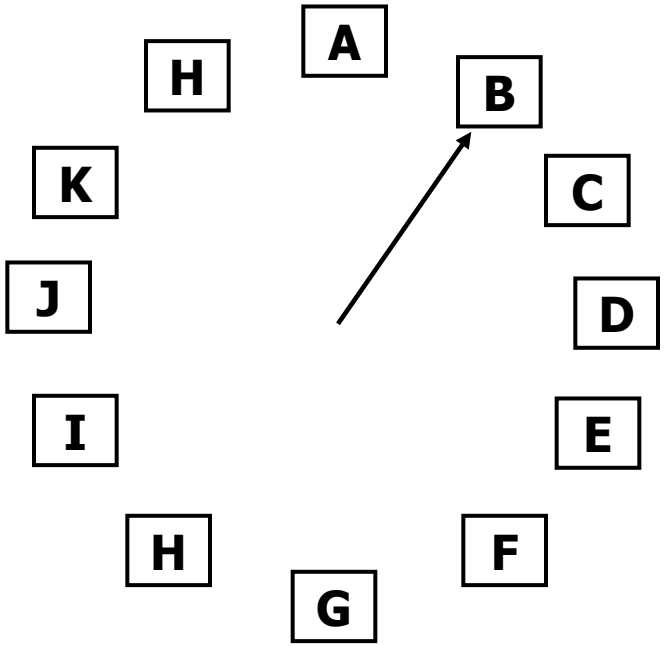


图5-24　时钟页面置换算法

5.9 页面置换算法

- Clock页面置换算法
  - 【例】假定系统为某进程分配了3个物理块，并考虑以下的页面引用串：7，0，1，2，0，3，0，4，2，3，0，3，2，1，2，0，1，7，0，1（20个页面请求）

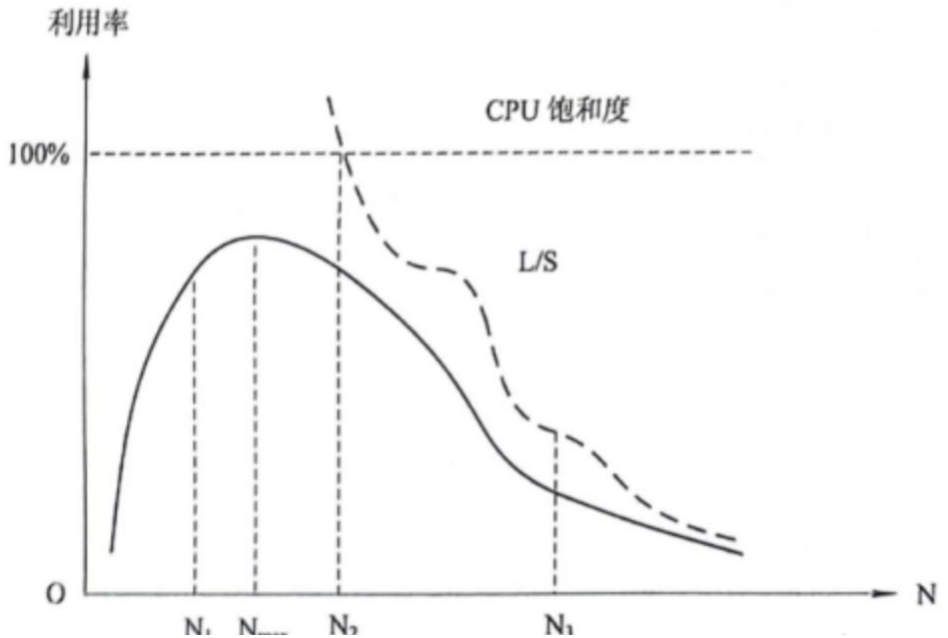
引用串	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
物理块	7*	7*	7*	2*	2*	2*	2*	4*	4*	4*	4	3*	3*	3	3	0*	0*	0	0*	0*
		0*	0*	0	0*	0	0*	0	2*	2*	2	2	2*	1*	1*	1*	1*	7*	7*	7*
			1*	1	1	3*	3*	3	3	3*	0*	0*	0*	0	2*	2*	2*	2	2	1*
被置换的页				7		1		2	0		3	4		2	0	3		1		2

图5-25　Clock页面置换算法的置换图

  - 发生11次页面置换，缺页率（11+3）/20

5.9 页面置换算法

- 抖动
  - 当进程需要将一个新页面调入内存时，因内存空间紧张，不得不将一个老页面置换出去，而刚刚置换出去的老页面可能又要被使用，需要重新将它调入。
  - 若一个进程频繁地进行页面调入调出，使系统效率降低，通常称这种现象为该进程发生了抖动。



5.9 页面置换算法

- 产生抖动的原因
  - 系统内的进程数太多，致使一个进程分得的内存块过少。
  - 系统采用的页面置换算法不合理等。
- 预防抖动的方法
  - 采取局部置换策略 ——将抖动限制在部分进程内
  - 处理机调度中引入工作集算法 ——合理分配物理块
  - 利用L=S准则调节缺页率 ——缺页率控制在合适范围
  - 暂停某些进程 ——释放部分内存资源

5.9 页面置换算法

- 工作集
  - 工作集是指进程在某段时间间隔△里，实际所要访问页面的集合。
  - 一个进程在时刻t-△到时刻t之间所访问的页面的集合称为该进程在时刻t的工作集 ，用W(t, △)表示。变量△称为“工作集窗口尺寸”，记为|W(t, △)|。
  - 例：对于如下页面引用序列，当窗口尺寸为3和4时，在T6时刻的工作集为（　　）。

时刻	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
引用页	1	4	3	2	1	5	4	3	2	2

A. {1,4,5}和{1,2,4,5}　B. {1,2,5}和{2,3,4,5}

C. {1,2,5}和{1,2,3,5}　D. {1,4,5}和{2,3,4,5}

5.9 页面置换算法

- 抖动的预防方法
  - 处理机调度中引入工作集算法
    - 处理机调度中调度新作业前，检查已有进程的工作集在主存的情况，如果内存中驻留的工作集页面不够，先不调度新作业。
    - 系统参考|W(t，△)|的大小来分配主存块，就既能有效地利用主存，又可以使缺页中断尽量少地发生。
  - 利用L=S准则调节缺页率
    - L：缺页之间的平均服务时间
    - S：平均缺页服务时间

5.9 页面置换算法

- 请求分页系统有效访问时间（EAT）计算
  - 参数：
    - λ 为访问快表的时间/更新快表的时间。
    - t 为访问内存的时间。
    - ε 为缺页中断处理时间
  - 页在内存，且快表检索命中

    EAT= λ + t
  - 页在内存，但快表检索没有命中

    EAT

    = 快表检索时间+访问页表时间+修改更新快表时间+访问页面物理内存时间

    = λ +t+ λ +t=2*(λ + t)

5.9 页面置换算法

- 请求分页系统有效访问时间（EAT）计算
  - 页面不在内存

EAT=快表检索时间+访问页表时间+缺页中断处理时间+修改更新快表时间+访问页面物理内存时间

$= \lambda + t + \varepsilon + \lambda + t$

  - 加入概率的情况(a是快表命中率，f是快表未命中下的缺页率)
  - EAT =快表检索时间+{a*访问页面物理内存时间+

(1-a)*[访问页表时间+ f*(缺页中断处理时间+更新快表时间+访问页面物理内存时间）+(1-f)*(更新快表时间+访问页面物理内存时间)]}

  - $=\lambda +\{a*t +(1-a)*[t + f*(\varepsilon+\lambda+t) +(1-f)*(\lambda+ t)]\}$

5.10 请求分段内存管理

- 段表
  - 由于应用程序的段，只有一部分装入内存，故需在段表中增加若干项，以供程序在调进、调出时参考。

段名	段长	段的基址	存取方式	访问字段A	修改位M	存在位P	增补位	外存始址

5.10 请求分段内存管理

- 地址变换

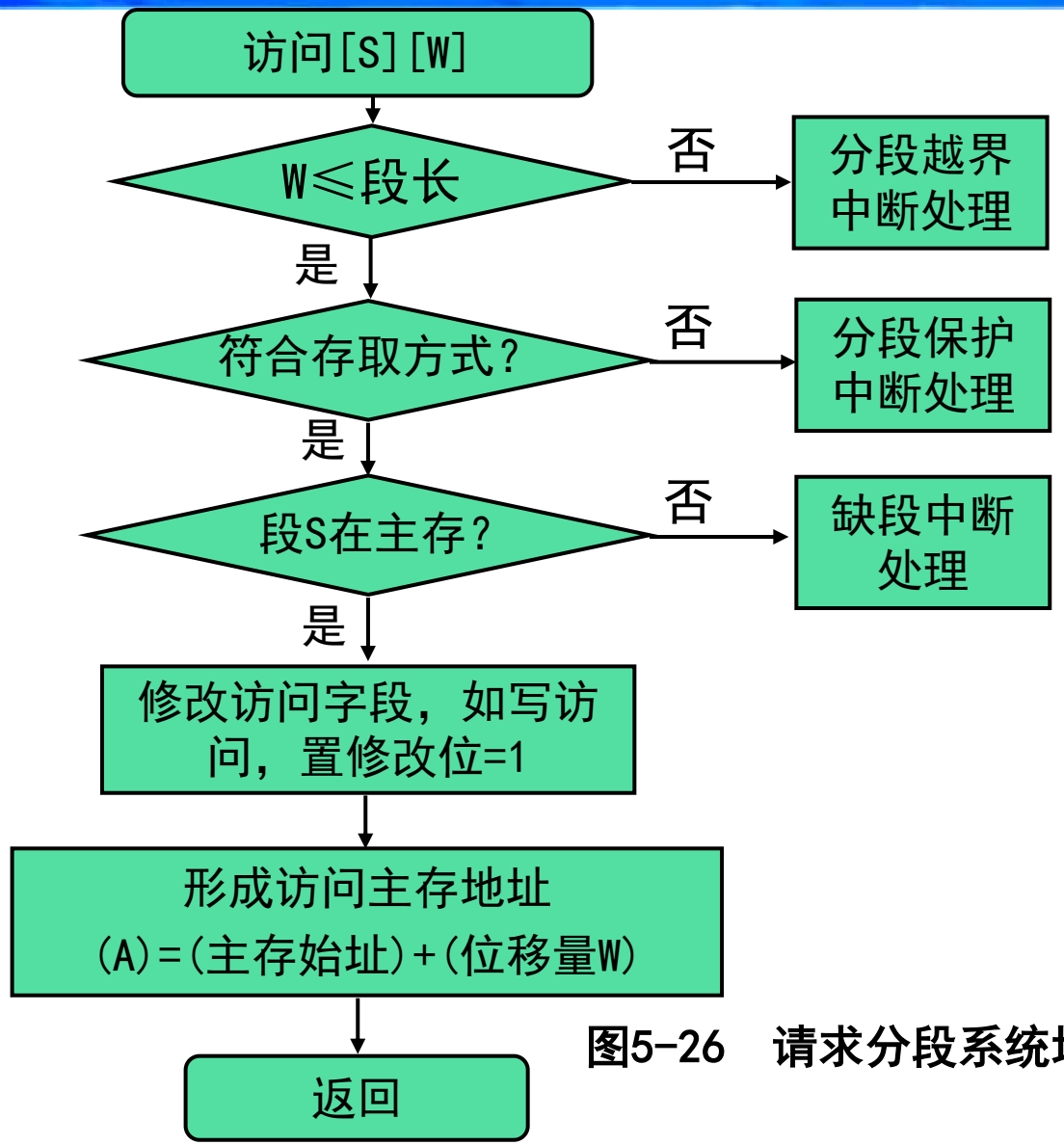


图5-26　请求分段系统地址变换

5.10 请求分段内存管理

- 缺段中断处理

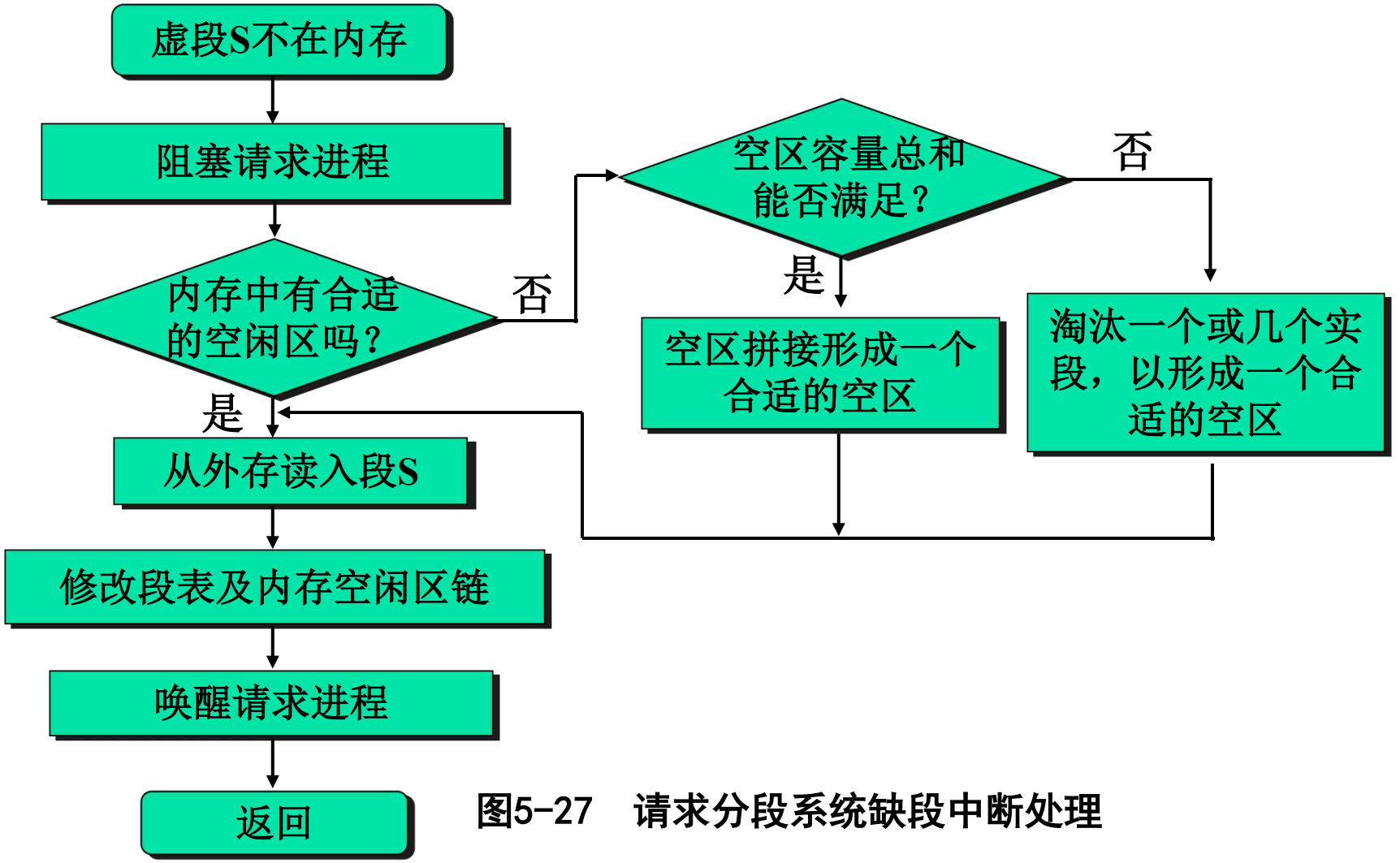


图5-27　请求分段系统缺段中断处理

5.10 请求分段内存管理

- 段的共享
  - 共享段表
    - 系统中设置一张共享段表。所有各共享段都在共享段表中占一表项。表项中记录了共享段的段号、段长、内存始址等信息，并记录了共享此段的每个进程的情况。

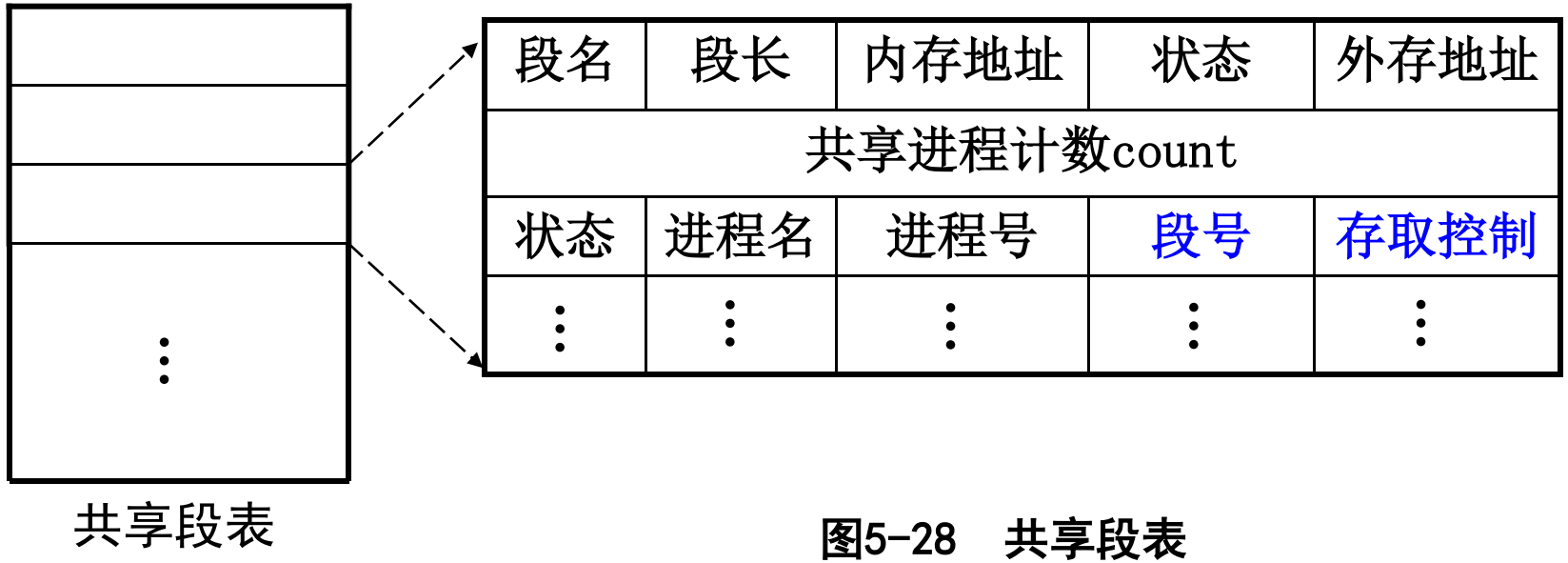


图5-28　共享段表

5.10 请求分段内存管理

- 共享段的分配
  - 对第一个请求使用该段的进程
    - 为该共享段分配一物理内存区
    - 更新请求进程段表信息
    - 在共享段表中增加一个表项，填写有关数据，count置1
  - 其它进程需要该段
    - 更新请求进程段表信息
    - **在共享段表中，填上调用进程的进程名、存取控制等；执行count++操作**
- 共享段的回收
  - **撤消该进程段表中该共享段对应的表项**
  - **count--；共享段表中该共享段的表项中去掉调用进程的项**
  - **当count=0时：取消共享段表中该段对应的表项，系统回收该段的内存；**